

UNIWERSYTET ZIELONOGÓRSKI

Wydział Nauk Inżynieryjno-Technicznych

Instytut Sterowania i Systemów Informatycznych

Praca magisterska

Kierunek: Informatyka

Specjalność: Inżynieria Systemów Informatycznych

Studia stacjonarne

UNIwersalny STEROWNIK SMART HOME

W OPARCIU O PROTOKÓŁ ZIGBEE

UNIVERSAL SMART HOME CONTROLLER BASED ON THE

ZIGBEE PROTOCOL

inż. Mateusz Banaszkiewicz

Promotor:

dr inż. Kamil Mielcarek

Pracę akceptuję:

(data i podpis promotora)

Zielona Góra, czerwiec 2025

Streszczenie

Niniejszy dokument opisuje proces projektowania i realizacji uniwersalnego sterownika przeznaczonego do systemów inteligentnego domu (Smart Home), bazującego na protokole komunikacyjnym Zigbee. Proponowane rozwiązanie różni się od istniejących produktów komercyjnych poprzez pełną lokalność, otwartość oraz możliwość rozbudowy i dostosowania funkcjonalności do indywidualnych potrzeb użytkownika, co stanowi odpowiedź na ograniczenia i ryzyka związane z aktualnymi trendami rynkowymi.

W pracy zaprezentowano autorskie podejście do konstrukcji urządzenia, poparte analizą rynku, przeglądem dostępnych rozwiązań oraz identyfikacją najczęstszych problemów zgłaszanych przez użytkowników. Przeprowadzono porównanie kluczowych cech, takich jak koszt, fizyczny rozmiar, zużycie energii, poziom bezpieczeństwa oraz kompatybilność ekosystemowa. Na tej podstawie opracowano i wdrożono prototyp sterownika, wyposażony w bibliotekę wspierającą integrację z różnymi typami urządzeń końcowych.

Przeprowadzone badania i testy wykazały, że możliwe jest stworzenie otwartego, funkcjonalnego i ekonomicznego rozwiązania Smart Home, które zachowuje wysoką elastyczność przy jednoczesnym ograniczeniu zagrożeń związanych z prywatnością i vendor-lockiem. Projekt stanowi wkład w rozwój niezależnych, lokalnych i rozszerzalnych rozwiązań w dziedzinie automatyki domowej.

Słowa kluczowe: Smart Home, Zigbee, sterownik, IoT, interoperacyjność, rozwiązania otwarte.

Spis treści

1. Wstęp	1
1.1. Idea i cel pracy	1
1.2. Zakres pracy	1
1.3. Motywacja i znaczenie tematu	2
2. Przegląd rozwiązań dostępnych na rynku	3
2.1. Czym jest Smart Home	3
2.1.1. Definicja	3
2.1.2. Przykładowe zastosowania	3
2.2. Przegląd statystyczny	5
2.2.1. Adopcja inteligentnych domów	5
2.2.2. Popularne rozwiązania	7
2.2.3. Opinie użytkowników	8
2.2.4. Podsumowanie	10
2.3. Parametry gotowych rozwiązań	11
2.3.1. Zestawienie ogólne	11
2.3.2. Zużycie energii	12
2.3.3. Rozmiar	12
2.4. Zagrożenia bezpieczeństwa i prywatności	12
2.4.1. Zagrożenia Wi-Fi [1, 2]	13
2.4.2. Szpiegowanie przez producentów	15
3. Projekt uniwersalnego sterownika	17
3.1. Czym jest sterownik uniwersalny	17
3.2. Protokoły komunikacyjne	17
3.2.1. Parametry ogólne	17
3.2.2. Zabezpieczenia oraz najczęstsze ataki	19

3.2.2.1.	Bluetooth LE	19
3.2.2.2.	Thread	21
3.2.2.3.	Matter	22
3.2.2.4.	Z-Wave	23
3.2.2.5.	Zigbee	24
3.2.2.6.	Zestawienie wyników	24
3.2.3.	Topologia siatki (Mesh) w Zigbee	26
3.2.4.	Kluczowe pojęcia protokołu Zigbee	27
3.3.	Wybór płytki	28
3.3.1.	Gotowe rozwiązanie	28
3.3.2.	Własne rozwiązanie	29
3.3.3.	Przegląd gotowych rozwiązań	30
3.3.3.1.	Gotowe uniwersalne mikrokontrolery	31
3.3.3.2.	Dedykowane układy SoC	33
3.3.3.3.	Moduły konsumenckie/OEM ze wsparciem Zigbee	34
3.3.3.4.	Platformy hybrydowe/mostowe	35
3.3.4.	Podsumowanie	37
3.4.	Środowisko programistyczne	37
4.	Testy oraz wyniki	39
4.1.	Platforma testowa	39
4.2.	Przeprowadzone testy	39
4.2.1.	Test platformy testowej z wykorzystaniem gotowych urządzeń	40
4.2.2.	Test przykładu HA_on_off_light	40
4.2.3.	Test trzech urządzeń	40
4.2.4.	Test przekaźników	41
4.2.5.	Test przełącznika	44
4.2.6.	Test potencjometru	45
4.2.7.	Test zasilania z baterii	46
4.2.8.	Test dwóch czujników	47
4.2.9.	Test stacji czujników z wyświetlaczem	47
4.2.10.	Test maksymalnego zasięgu	50
4.2.11.	Test zużycia energii przez telefon	51

4.3.	Propozycje testów	52
4.3.1.	Test kompatybilności	52
4.3.2.	Test szyfrowania	52
4.3.3.	Test zabezpieczeń Frame + Nonce	53
4.3.4.	Test implementacji	53
4.4.	Porównanie osiągnięć	53
4.5.	Napotkane problemy	56
4.5.1.	Początkowe przygotowania	56
4.5.2.	Konfiguracja środowiska	56
4.5.3.	Integracja Zigbee SDK	57
4.5.4.	Testowanie i debugowanie	58
5.	Analiza otrzymanych wyników w kontekście badań	59
5.1.	Podsumowanie wyników	59
5.2.	Ograniczenia proponowanego rozwiązania	60
5.3.	Możliwości dalszego rozwoju	61
5.4.	Wnioski końcowe	63

Spis rysunków

2.1. Porównanie wskaźnika adopcji Smart Home w różnych regionach (2024)	6
2.2. Szacowana i prognozowana liczba globalnych użytkowników systemów Smart Home w latach 2023-2027	6
2.3. Procent gospodarw USA posiadających konkretne urządzenia w roku 2024.	7
2.4. Najpopularniejsze opinie użytkowników.	8
2.5. Najczęstsze problemy użytkowników.	9
2.6. Zrzut ekranu z podglądu w taniej kamerze. Źródło: recenzja użytkow- nika w serwisie Allegro.	10
2.7. Obraz z kamery Ring opanowanej przez hakera	14
3.1. Struktura sieci Mesh [3]	26
4.1. Funkcja esp_zb_task zmodyfikowana do deklaracji trzech endpointów	41
4.2. Schemat układu zastosowanego w teście przekaźników.	42
4.3. Zdjęcie faktycznego układu zastosowanego w teście przekaźników. . .	42
4.4. Fragment funkcji zb_attribute_handler w teście przekaźników. . . .	43

Spis tabel

2.1. Najpopularniejsze urządzenia Smart Home - udział rynkowy i liczba egzemplarzy (2023–2024)	7
2.2. Zestawienie wymiarów, cen i zużycia energii przykładowych smart urządzeń.	11
3.1. Porównanie: Wi-Fi, Bluetooth LE i Zigbee [4, 5, 6, 7, 8, 9, 10]	18
3.2. Porównanie: Z-Wave, Thread i Matter [11, 12, 13, 14, 15]	18
3.3. Zestawienie omówionych zagrożeń i zabezpieczeń rozważanych protokołów komunikacyjnych.	25
4.1. Zestawienie zużycia mocy przez sterownik uniwersalny w teście przekazników.	44
4.2. Zestawienie zużycia mocy przez sterownik uniwersalny w teście zasilania z baterii.	46
4.3. Zestawienie zużycia mocy przez sterownik uniwersalny w teście stacji czujników z wyświetlaczem.	49
4.4. Porównanie typowych urządzeń końcowych Smart Home względem modułu ESP32-C6	55

Rozdział 1

Wstęp

1.1. Idea i cel pracy

Celem pracy jest stworzenie uniwersalnego modułu z protokołem komunikacyjnym Zigbee. Posiada on pełną integrację z systemami Smart Home, co pozwala na wykorzystanie go do obsługi wielu urządzeń peryferyjnych takowej integracji nie posiadających. Moduł ten ma oferować alternatywę dla dostępnych rozwiązań, osiągając lepsze bądź porównywalne parametry, eliminując tym samym główne problemy związane z korzystaniem z gotowych urządzeń.

1.2. Zakres pracy

Praca obejmuje:

- przegląd i analizę rozwiązań dostępnych na rynku,
- ewaluację bezpieczeństwa i prywatności w kontekście rozważanych urządzeń,
- projekt oraz prototyp modułu,
- testy integracyjne z wykorzystaniem Home Assistant i Zigbee2MQTT,
- porównanie osiągnięć modułu do parametrów badanych urządzeń,
- podsumowanie obserwacji wraz z identyfikacją możliwości dalszego rozwoju.

1.3. Motywacja i znaczenie tematu

Inteligentne domy to systemy które są z dnia na dzień coraz bardziej popularne, a co za tym idzie, na rynku jest dostępnych wiele rozwiązań, oferujących bardzo szeroki wachlarz możliwości. Zważywszy na różne firmy chcące utrzymać swoich klientów, częstym zjawiskiem są zamknięte ekosystemy, które zmuszają do korzystania z jedynie kompatybilnych urządzeń, nierzadko kosztem wyższym niż u konkurencji. Przy tym chęć pozyskania jak największej ilości danych od użytkowników, bardzo często wiąże się z wymaganiem instalowania aplikacji, tworzenia kont, oraz gromadzenia informacji w chmurze. To co może się wydawać wygodne, jest potencjalnym ryzykiem utraty prywatności, mogącym prowadzić do znacznego zmniejszenia bezpieczeństwa osób korzystających z tych systemów.

Uniwersalny moduł może się okazać odpowiedzią na wiele z tych problemów, a poszczególne aspekty implementacji są w stanie przewyższyć oczekiwania, przebijając osiągnięcia dużo bardziej kosztownych rozwiązań.

Rozdział 2

Przegląd rozwiązań dostępnych na rynku

2.1. Czym jest Smart Home

2.1.1. Definicja

Jak definiuje Ablowitz: „*Smart home integruje technologie komputerowe i komunikacyjne do monitorowania i sterowania systemami domowymi — lokalnie lub zdalnie — w celu poprawy jakości życia i zarządzania zasobami.*” [16] Innymi słowy, inteligentny dom to system, który za pomocą sieci czujników i modułów wykonawczych automatyzuje pewne procesy, w oparciu o reguły określone przez użytkownika.

2.1.2. Przykładowe zastosowania

Systemy Smart Home mają szeroki wachlarz zastosowań, usprawniając lub rozwijając niektóre aspekty życia codziennego. Poniżej przedstawione zostały wybrane przykłady, podzielone na obszary, które cieszą się największym zainteresowaniem wśród różnych grup użytkowników. Lista została utworzona w oparciu o propozycję Ablowitza [16].

Komfort i wygoda

Urządzenia przeznaczone do automatyzacji codziennych czynności, które wymagają od użytkownika manualnej obsługi bądź przemieszczania się.

- Oświetlenie - Inteligentne żarówki lub taśmy LED umożliwiają pełną kontrolę nad barwą i natężeniem światła. Możliwe jest automatyczne dopasowanie parametrów do warunków panujących w mieszkaniu, takich jak obecności osób w pokoju czy zachmurzenia wywołującego półmrok.
- Sterowanie głosowe - Asystenci głosowi tacy jak Google Assistant bądź też Amazon Alexa wspierają wiele systemów, umożliwiając bezdotykowe zarządzanie.

Zarządzanie energią

Urządzenia zapewniające informacje o zużyciu prądu oraz podejmujące decyzje na bazie danych z czujników w celu jego zmniejszenia.

- Ogrzewanie i klimatyzacja - Czujniki obecności w połączeniu z inteligentnymi termostatami pozwalają na zautomatyzowanie pracy urządzeń zarządzających temperaturą. System może samodzielnie wyłączyć ogrzewanie po wykryciu braku obecności domowników, oszczędzając tym samym energię.
- Monitorowanie zużycia prądu - Inteligentne gniazdka oferują stały pomiar poboru energii przez podpięte do nich urządzenia, co pozwala zidentyfikować zbędne wydatki. Dodatkowo umożliwiają wyłączanie przepływu prądu, np. w godzinach nocnych.

Bezpieczeństwo

Systemy oferujące zabezpieczenia przed włamaniami lub informujące o nietypowych zdarzeniach, mogących naruszać prywatność bądź bezpieczeństwo.

- Monitoring i systemy alarmowe - Wszelkiego rodzaju czujniki wykrywające ruch oraz obecność osób w mieszkaniu, w połączeniu z kamerami pozwalają na pełny wgląd do mieszkania, również zdalnie. Dodatkowo inteligentne czujniki dymu oraz czadu mają przewagę nad tradycyjnymi, ze względu na stały podgląd w stan baterii oraz sprawności urządzenia.
- Symulowanie obecności - W przypadku nieobecności mieszkańców, możliwe jest okresowe włączanie różnych sprzętów, takich jak światła czy muzyka w celu odstraszenia potencjalnych włamywaczy, którzy nie będą wiedzieli, że dom w rzeczywistości jest pusty.

Multimedia i integracja

Urządzenia ułatwiające zarządzanie inteligentnym domem bądź w inny sposób zapewniające prostotę obsługi i rozszerzenie możliwości.

- Wyświetlacze - Rozmieszczenie takich urządzeń w strategicznych miejscach pozwala na ułatwiony dostęp do systemu oraz zmniejsza ryzyko niezauważenia istotnych powiadomień.
- Głośniki - Podobne do wyświetlaczy, lecz zapewniające również możliwość odtwarzania różnych mediów na żądanie.
- Telewizory - Smart TV umożliwiają integrację z asystentami głosowymi oraz urządzeniami mobilnymi, co pozwala na wygodne odtwarzanie multimediów oraz sterowanie głosem lub telefonem.

Ten podział pozwoli na większe zrozumienie podczas zagłębiania się w liczby i zestawienia, podkreślając trendy oraz wyjaśniając pewne schematy.

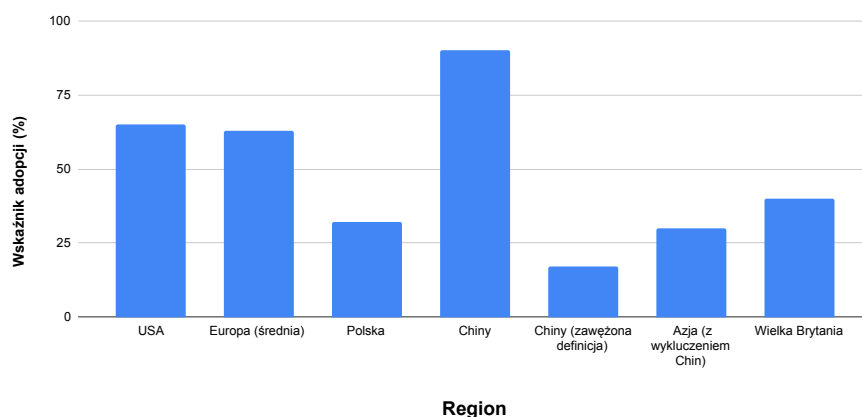
2.2. Przegląd statystyczny

W tym punkcie zostały opisane statyski, zawierające informacje o ilości inteligentnych domów (tzn. gospodarstw domowych z przynajmniej jednym urządzeniem Smart Home) na świecie, oraz które zagadnienia interesują konsumentów, a które wciąż wiążą się z obawami bądź zgłaszanymi problemami.

2.2.1. Adopcja inteligentnych domów

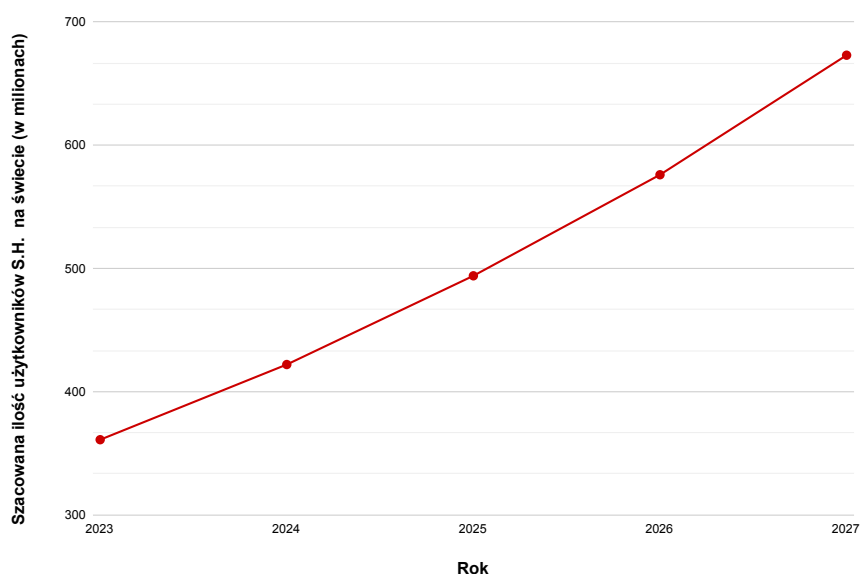
Systemy te z pewnością są bardzo popularne. Według statystyk przedstawionych na wykresie 2.1, Stany Zjednoczone są inteligentne w 65 procentach, natomiast w Chinach ponad 90% mieszkań posiada urządzenia Smart Home. Interesującym jest to, iż tak wysoki odsetek skutkował bardzo luźną definicją kryteriów. Do modułów, które zaliczały się jako spełniające warunki należały takie urządzenia jak Smart TV czy nawet telefon. Zawężając grono odpowiedzi do tych uwzględniających tylko rodzaje automatyki domowej, ta liczba spadła aż do 17 procent.

Przedstawione dane są szacunkowe, więc należy je traktować tylko jako wstępne rozpoznanie. Źródła: [17, 18, 19, 20, 21]



Rysunek 2.1. Porównanie wskaźnika adopcji Smart Home w różnych regionach (2024)

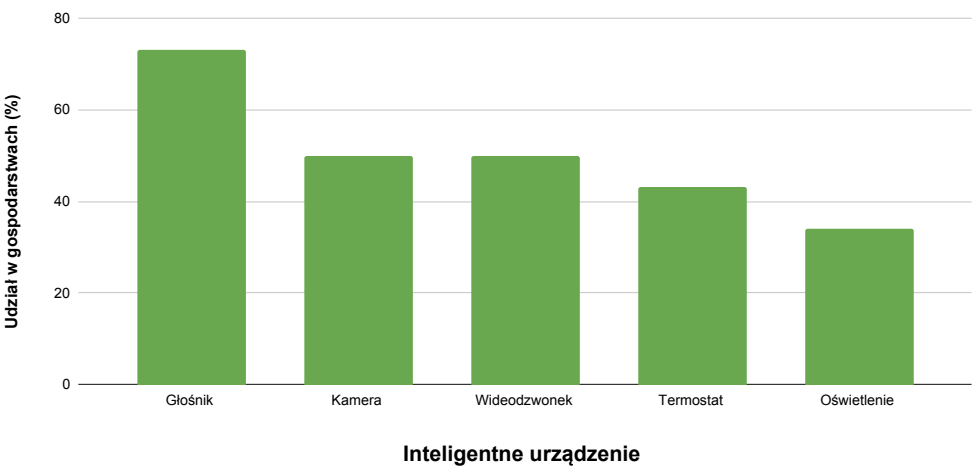
Według Strategic Market Research [20] ilość użytkowników inteligentnych domów z każdym rokiem znacząco wzrasta. Obecnie stanowi ona w przybliżeniu 494 miliony, czyli aż 16% ludzi na Ziemi. W roku 2027 ta liczba ma wzrosnąć do 673 milionów. Na wykresie 2.2 przedstawione zostały szacowane liczby w latach 2023-2027. Średnio, z roku na rok ilość użytkowników Smart Home na świecie wzrasta o 20-25%.



Rysunek 2.2. Szacowana i prognozowana liczba globalnych użytkowników systemów Smart Home w latach 2023-2027

2.2.2. Popularne rozwiązania

Na podstawie badań przeprowadzonych przez Morning Consult[22] stworzono wykres 2.3. Widać, iż największym zainteresowaniem cieszą się inteligentne głośniki, a połowa wszystkich domów w USA w 2024 roku była wyposażona w inteligentne kamery oraz dzwonki wideo.



Rysunek 2.3. Procent gospodarw USA posiadających konkretne urządzenia w roku 2024.

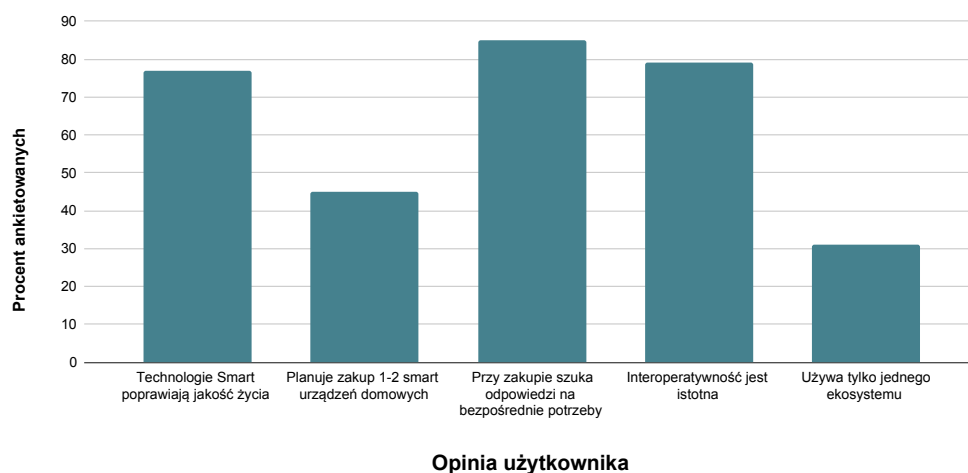
Urządzenie	Kategoria	Udział	Sztuki	Region	Rok	Źródło
Smart speaker	Multimedia, Sterowanie	73%	-	USA	2024	[22]
Smart speaker	Multimedia, Sterowanie	-	<200 mln	Globalnie	2024	[23]
Kamera	Bezpieczeństwo	50%	-	USA	2024	[22]
Kamera	Bezpieczeństwo	11%	-	EU	2023	[24]
Termostat	Energia, Komfort	43%	-	USA	2024	[22]
Robot sprzątający	Komfort	-	80 mln	Globalnie	2023	[25]
Oświetlenie	Komfort	34%	-	USA	2024	[22]
Oświetlenie	Komfort	14%	-	EU	2023	[24]
Smart TV	Multimedia	-	200 mln	Globalnie	2023	[26]
Gniazdka, przełączniki	Energia, Komfort	20%	-	Globalnie	2023	[26]
AGD (np. lodówki)	Komfort	<10%	-	Globalnie	2023	[27]

Tabela 2.1. Najpopularniejsze urządzenia Smart Home - udział rynkowy i liczba egzemplarzy (2023–2024)

W tabeli 2.1 umieszczono wybrane dane z podobnych badań konsumenckich, wykazujące procentowe występowanie we wszystkich gospodarstwach w regionie bądź sprzedane w tym okresie egzemplarze. Zestawienie wykazuje, iż inteligentne głośniki cieszą się zainteresowaniem porównywalnym do Smart TV, które są obecnie standardem na rynku telewizorów.

2.2.3. Opinie użytkowników

Aby lepiej zrozumieć jak wcześniej wymienione liczby przekładają się na rzeczywistość, dokonano analiz ankiet przeprowadzonych przez HealthPopuli[28] oraz Deloitte[27]. Wnioski z nich wyciągnięte będą istotną zmienną w procesie projektowania uniwersalnego sterownika.

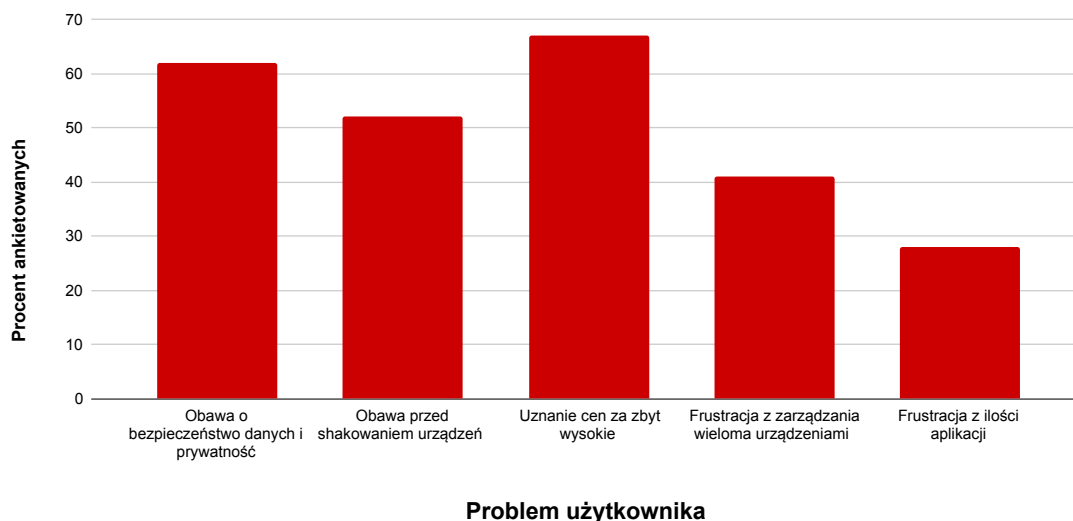


Rysunek 2.4. Najpopularniejsze opinie użytkowników.

Z danych przedstawionych na wykresie 2.4 można wyczytać, iż respondenci są wysoce zgodni co do tego, że technologie Smart poprawiają jakość życia. Zapewne właśnie ta idea jest motywacją przy zakupach.

Kwestia interoperatywności jest bardzo istotna: potwierdza to prawie 80% ankietowanych. Mimo tak dużej liczby, tylko 30% zamyka się w jednym, konkretnym systemie. To pokazuje jak istotna jest uniwersalność, oraz kompatybilność różnych modułów od różnych producentów.

Warto również wspomnieć, iż oba badania potwierdzają, że osoby posiadające już jedno urządzenie, są o wiele bardziej przychylne zakupie następnego.



Rysunek 2.5. Najczęstsze problemy użytkowników.

Na wykresie 2.6 przedstawione zostały najczęściej występujące problemy, które hamują potencjalnych nowych klientów, bądź frustrują obecnych. Dominującą jest kwestia cen inteligentnych produktów: aż 67% pytanych uważa, iż są one za wysokie. Analizując rynek, można stwierdzić, iż te odczucia nie są bezpodstawne. Poniżej przedstawione zostały przykładowe koszty, na jakie trzeba się przygotować, kupując inteligentne urządzenia. Badania wykonano w popularnym w Polsce serwisie Allegro. Dane pochodzą z czerwca 2025 roku.

- Czujnik ruchu: 32 - 87 PLN.
- Żarówka RGB: 23 - 118 PLN.
- Termostat: 126 - 270 PLN.
- Centralny HUB: 65 - 440 PLN.
- Kamera: 60 - 280 PLN.
- Czujnik dymu/czadu: 40 - 200 PLN.

Warto pamiętać, iż cena wiąże się z jakością, jakiej można oczekiwać. Np: według opinii kupujących tanie żarówki bardzo szybko się przepalały, czujniki często nie wykrywały ruchu, a kamery były praktycznie ślepe w nocy, mimo wbudowanej podczerwieni i lamp. Dodatkowo ważną jest kwestia bezpieczeństwa i prywatności,

omówiona w punkcie 2.4. Zaufane marki zazwyczaj cenią swoją renomę, co sprawia, że aby czuć się bezpiecznie, należy odpowiednio zapłacić.

Kupując inteligentny produkt należy również zapoznać się z jego specyfikacją. Możliwym jest, iż do jego obsługi potrzebny jest konkretny hub producenta (tak zwany vendor lock-in), bądź inne urządzenie, bez którego nie będzie można z niego korzystać. Jest to kolejny istotny dla konsumentów aspekt. Zbyt wiele marek, które nie działają jedna z drugą prowadzi do frustracji z ilości aplikacji i obsługi różnych urządzeń.



Rysunek 2.6. Zrzut ekranu z podglądu w taniej kamerze. Źródło: recenzja użytkownika w serwisie Allegro.

2.2.4. Podsumowanie

Inteligentne urządzenia są w dużym stopniu integralne w gospodarstwach na całym świecie przy czym prym wiodą inteligentne głośniki. Obszarami cieszącymi się największą popularnością są komfort oraz bezpieczeństwo.

Z odpowiedzi ankietowanych można wywnioskować, iż znacząca większość użytkowników Smart urządzeń jest zadowolona z zakupu, choć przy mieszaniu różnych producentów obsługa jest frustrująca, głównie z powodu ilości aplikacji. Duża część populacji myśli o zagłębieniu się w tematykę inteligentnych domów, szukając rozwiązań na dręczące ich problemy, lecz hamują ją przed tym wysokie ceny urządzeń, oraz obawy o bezpieczeństwo swoich danych i prywatność.

Przy zakupie inteligentnych urządzeń należy liczyć się z dość wygórowanymi cenami. Stawiając na produkty tanie, konsument jest narażony na słabej jakości sprzęt o krótkiej żywotności, bez kompatybilności z innymi modułami inteligentnych domów oraz potencjalnie kompromitujący prywatne dane.

2.3. Parametry gotowych rozwiązań

Cena oraz kompatybilność urządzenia to nie wszystko. Istotne są również jego fizyczne wymiary, a także zużycie energii. Nawet najlepszy termometr nie będzie cieszył się zainteresowaniem, jeśli będzie zajmował za dużo miejsca i wymagał częstej wymiany baterii. W tej sekcji przedstawione zostały przykłady urządzeń, które można obecnie zakupić.

2.3.1. Zestawienie ogólne

Urządzenie	Protokół	Zużycie energii	Wymiary (mm)	Cena (pln)	Źródło
Sonoff ZBMINI Przełącznik	Zigbee 3.0	-	42.6 × 42.6 × 20	27.52	[29]
Shelly 1 Gen3 Przełącznik	Wi-Fi / Bluetooth	< 1.2W	37 × 42 × 16	71.90	[30]
Fibaro Single Switch 2 Przełącznik	Z-Wave	-	42.5 × 38.25 × 20.3	167.79	[31]
Aqara Temp Sensor Czujnik temperatury	Zigbee (Aqara Hub)	Bateria CR2032	36 × 36 × 9	44.83	[32]
Shelly H&T Czujnik temp. i wilg.	Wi-Fi	Bateria CR123A ~18 miesięcy	46 × 35 (Ø × H)	99	[33]
Fibaro Motion Sensor Czujnik ruchu	Z-Wave Plus	Bateria CR123A	45 × 60 (Ø × H)	139	[34]
Philips Hue Bulb E27 Żarówka LED	Bluetooth / Zigbee	max 0.5W standby max 9W Aktywna	60 × 110 (Ø × H)	49.99 2 sztuki	[35]
Nest Protect Czujnik dymu/czadu	Wi-Fi / Bluetooth	6 × Bateria AA	135 × 135 × 39	636	[36]
Gosund SP111 Smart gniazdko	Wi-Fi	-	79 × 48 × 46	45	[37]

Tabela 2.2. Zestawienie wymiarów, cen i zużycia energii przykładowych smart urządzeń.

Dane w tabeli pochodzą z ofert znalezionych na polskim rynku. W przypadku braku informacji o zużyciu prądu bądź wymiarach urządzeń, podejmowano próbę ich lokalizacji na stronach producentów. Mimo to często trudno było znaleźć konkretne liczby dotyczące zużycia energii.

2.3.2. Zużycie energii

Aby mieć odwołanie co do wielkości liczb oznaczających zużycie prądu wykonano test 4.2.11. Wynika z niego, iż telefon pozostawiony w trybie czuwania z włączonymi protokołami Wifi, Bluetooth i GPS osiągnął zużycie prądu 0.12W. Innym przykładem może być monitor 4K Gigabyte M28U, który w trybie czuwania zużywa 0.5W. To są typowe wyniki osiągane przez duże urządzenia, wykonujące dużo czynności i zadań wymagających wiele energii. Czujniki, które mają jedynie okresowo przysyłać informacje o np. temperaturze otoczenia nie powinny przekraczać takich liczb.

2.3.3. Rozmiar

W przypadku wielu z rozważanych urządzeń, ich fizyczne wymiary nie mają zbyt wielkiego znaczenia. Jednak wartym uwagi są np. przełączniki, które to użytkownicy chętnie montują za przełącznikami światła. Zbyt wielkie urządzenie może ograniczać miejsca, w których można je zastosować, co jest jedną ze zgłaszanych wad Sonoff ZBMINI, gdzie najlepiej oceniana recenzja zgłasza problemy z instalacją modułu z powodu jego wielkości.

2.4. Zagrożenia bezpieczeństwa i prywatności

Inteligentny dom z założenia ma przejąć wiele obowiązków, których użytkownik nie chce robić samemu, zapewniając mu wygodę. Jak wynika ze statystyk, wiele osób powierza też tym systemom wgląd we własne mieszkania za pomocą inteligentnych kamer, bądź przeróżnych czujników. Każde z tych rozwiązań z pewnością przynosi wiele korzyści dostarczając wielu informacji lub mogąc zarządzać ważnymi systemami. Jednak może się pojawić problem, gdy posiadacz systemu nie jest jedyną osobą mającą dostęp do tych danych i opcji.

2.4.1. Zagrożenia Wi-Fi [1, 2]

Jeżeli domowa sieć Wi-Fi zostanie skompromitowana, atakujący może z łatwością mieć dostęp do systemu oraz poszczególnych urządzeń. Jednak taki scenariusz nie zdarza się zbyt często. W przypadku rozwiązań IoT komunikujących się po Wi-Fi, można wykorzystać ich słabe strony, takie jak:

- **Niepotrzebnie otwarte porty:** jeśli urządzenie IoT nasłuchuje na portach, które nie są wymagane do ich pełnego działania, atakujący może próbować się do nich podpiąć i wysłać szkodliwy kod.
- **Zbyt słabe uwierzytelnienie lub jego brak:** często aby móc podłączyć się do urządzenia należy podać hasło. Jednak nie każde urządzenie wymaga od użytkownika jego zmiany na silniejsze niż fabryczne. W rezultacie atakujący który zna producenta i model urządzenia może z powodzeniem odgadnąć hasło i uzyskać pełny dostęp.
- **Brak szyfrowania:** jeżeli przesył danych z i do systemu nie jest w żaden sposób szyfrowany, bądź używa zbyt prostego algorytmu, te dane może podsłuchać każdy kto jest w zasięgu sygnału (Na przykład na chodniku przed domem ofiary).
- **Zbyt rzadkie aktualizacje bądź ich brak:** luki w obronie są nieuniknione. Jeśli producent znalazł sposób na zwalczenie jednej z nich, wydaje aktualizację. Jeśli użytkownik nie utrzymuje swoich systemów i urządzeń w zgodzie z najnowszymi wersjami oprogramowania, atakujący może ciągle wykorzystać stare słabości, nawet bardziej uwydatnione, po wprowadzeniu aktualizacji, która zwróciła na nie uwagę.

Oczywiście każdemu z tych problemów można przeciwdziałać, stosując odpowiednie kroki, takie jak częste aktualizacje i ustawienie silnego hasła, niemniej ważnym jest zrozumienie, iż połączenie bezprzewodowe nigdy nie jest w stu procentach bezpieczne. Dlatego warto się zastanowić, czy korzystanie z urządzeń dających dostęp do bardzo prywatnych informacji (na przykład kamery z podglądem video do środka domu) musi być koniecznie bezprzewodowe, a jeśli tak, należy zainwestować w wysokiej jakości sprzęt, odporny na ataki.

Kolejnym faktem jest to, iż każde urządzenie w sieci, jeśli skompromitowane, może dać atakującemu dostęp do całego systemu. Najlepszym rozwiązaniem by uniknąć tego typu przykrości, jest korzystanie z jak najmniejszej liczby urządzeń podłączonych do internetu, wybierając inne protokoły, które nawet gdy zostaną złamane, nie zapewnią żadnych prywatnych informacji zagrażających bezpieczeństwu. Inteligentny termometr nie musi korzystać z Wi-Fi by przekazywać dane o temperaturze.

Kilka przykładów ilustrujących faktyczne ataki, wykorzystujące opisane słabości:

- **Atak na inteligentne kamery Ring w roku 2020 [38]**

Hakerzy znaleźli lukę w oprogramowaniu, dzięki czemu uzyskali zdalny dostęp do urządzeń. Mogli na żywo oglądać klientów Ring'a w ich własnych domach bez ich wiedzy, lub też nękać ich, mówiąc przez głośniki. Wiele osób zgłaszało groźby, rasistowskie wyzwiska oraz próby szantażu. Firma Ring została pozwana za "prerażająco niskie zabezpieczenia". W odpowiedzi padły argumenty o zbyt słabych hasłach i nie korzystanie z dwustopniowej weryfikacji.



Rysunek 2.7. Obraz z kamery Ring opanowanej przez hakera

- **Oprogramowanie inteligentnego łóżka Eight Sleep umożliwiające dostęp do całej sieci [39, 40]**

W grudniu 2024 użytkownik portalu X @InsecureNature opublikował wpis,

gdzie poinformował o jednym z jego domowych sprzętów posiadającym jego klucz AWS (Amazon Web Services), który jest bardzo prywatną informacją. Dodatkowo, zaraz po tym odkryciu dokonał kolejnego. Programiści pozostawili w kodzie furtkę do przejęcia kontroli nad komputerem urządzenia. Pozwalała ona na dostęp spoza lokalnej sieci i dawała takie informacje jak w jakich godzinach i ile osób spało w łóżku, a co za tym idzie wykrycie kiedy łóżko jest puste, mimo pory nocnej, co mogło informować o braku mieszkańców w domu. Inny badacz opisał później tą furtkę jako bramę do wszystkich pozostałych urządzeń w sieci, w tym telefonów, komputerów, inteligentnych zamków i kamer.

2.4.2. Szpiegowanie przez producentów

Kompromitacja systemu przez strony trzecie jest zawsze zagrożeniem, ale można podjąć kroki które minimalizują to ryzyko. Zupełnie inną jest sytuacja, gdy użytkownik jest szpiegowany od samego początku, przez samego producenta.

Połączenie z internetem jest wygodne by móc wygodnie skonfigurować urządzenie, jednak nigdy nie można mieć pewności, czy dane nie są wysyłane również gdzieś indziej. Jeśli cały system Smart Home jest oparty na działaniu w chmurze, nie wiadomo, w jaki sposób te dane są używane.

Wyżej wymieniony przykład łóżka Eight Sleep świetnie zobrazował do czego zdolni są producenci. Oto kilka kolejnych przykładów:

- **Google Assistant nagrywający rozmowy użytkowników [41]**

W roku 2017 Google przyznał się do tego, iż ich urządzenia z wbudowanym asystentem głosowym nagrywają dźwięk po tym jak rozpoznają (nawet jeśli fałszywie pozytywnie) komendę "Hey Google". Nie byłoby to dziwne w funkcjonowaniu programu, gdyby nie fakt o wysyłaniu tych nagrań, często zawierających prywatne rozmowy na serwery Google. Potwierdzonym jest przesłuchiwanie nagrań przez kontrahentów w celu lepszego zrozumienia wzorców językowych i akcentów. Mimo iż firma oznajmiła, że wszelkie prywatne informacje były kasowane, raport VRT wykazał, że znalezione zostały nagrania zawierające między innymi dane takie jak adres zamieszkania użytkownika, lub inne, gdzie osoba mówiła o swoim życiu intymnym. Wkrótce potem Google oświad-

czyło, że błąd oprogramowania pozwalał na nagrywanie nawet bez aktywacji asystenta. Jednak naiwną wydaje się wiara w prawdziwość tych zeznań.

- **Chińskie urządzenia kontaktujące się z zagranicznymi serwerami [42]**

W marcu 2021 firma Dark Cubed przetestowała 10 inteligentnych urządzeń sprzedawanych na amerykańskim rynku, w cenach wachających się od 20 do 100 dolarów. Każde z testowanych urządzeń okazało się kontaktować z chińskimi serwerami, bez zgody użytkownika. Wartym uwagi jest, iż chińska ustawa o bezpieczeństwie danych z 2021 r. wymaga od rodzimych firm współpracy z organami rządowymi. To daje wgląd tym urzędom we wszystkie dane, które zostały zebrane przez inteligentne urządzenia w domach klientów.

- **Systemy działające w chmurze**

Zbieranie danych wcale nie musi być ukryte. Każda usługa działająca w chmurze, jest oparta o dobrowolne wysyłanie danych na serwery dostawcy. Jeśli ten system zawiera urządzenia, takie jak: kamery, czujniki ruchu, czujniki obecności, mikrofony, ale także nawet oświetlenie, operator usługi będzie w stanie nie tylko mieć dostęp do każdego z nich, ale też będzie w stanie stworzyć harmonogram dnia konsumenta, oraz będzie wiedział kiedy dom jest pusty. To, co dzieje się z tymi danymi, zależy wtedy tylko od uczciwości firmy, co jest bardzo poważnym zagrożeniem prywatności i bezpieczeństwa.

Firmy oferujące usługi w chmurze, bądź otwarcie przyznające się do gromadzenia danych z urządzeń: Amazon (Alexa, Echo), (Google Home, Nest), Apple (Siri), Tuya, Xiaomi / Aigostar (kuchnia, RTV), Samsung, LG (smart TV), etc.

Alternatywą mogą być systemy działające w stu procentach lokalnie (np. Home Assistant, Domoticz). Własnoręcznie postawiony system, najlepiej na oddzielnej maszynie, odłączonej od internetu.

Rozdział 3

Projekt uniwersalnego sterownika

3.1. Czym jest sterownik uniwersalny

Sterownik uniwersalny to moduł, który jest kompatybilny z jak największą liczbą dostępnych ekosystemów Smart Home. Ma za zadanie przesłać systemowi informacje o tym czym jest, jakie informacje jest w stanie doręczyć, oraz jakie funkcje system może u niego wywołać.

Sterownik ten umożliwia użytkownikowi podłączenie do niego przeróżnych urządzeń które same w sobie nie są w stanie komunikować się z systemem Smart Home. Służy on więc jako swoisty most pomiędzy “nieinteligentnym” urządzeniem a jednostką centralną Smart Home. Dzięki uniwersalności, użytkownik sam może wybrać co chce do sterownika podłączyć, oraz zmienić układ, jeśli będzie chciał. To zapewnia nie tylko wsparcie w przypadku chęci zarządzania sprzętem który nie łączy się z systemem, ale także możliwość testowania potencjalnych pomysłów bez potrzeby zakupów wielu, często drogich gotowych rozwiązań.

3.2. Protokoły komunikacyjne

3.2.1. Parametry ogólne

Wybór właściwego sposobu na komunikację sterownika z jednostką centralną jest bardzo ważny. Każdy ma swoje wady i zalety, które mogą sugerować konkretne zastosowania. Tabele 3.1 i 3.2 przedstawiają informacje potrzebne do podjęcia właściwej decyzji.

Właściwość	Wi-Fi	Bluetooth LE	Zigbee
Częstotliwość	2.4 / 5 GHz	2.4 GHz	2.4 GHz (globalnie), Sub-GHz (niektóre regiony)
Zasięg	do 30 m (wewnątrz), do 100 m (na otwartej przestrzeni)	do 10–30 m (wewnątrz), do 100 m (na zewnątrz, punkt–punkt)	do 20 m na hop, >500 m w mesh (zależnie od liczby węzłów)
Główne zastosowania	Urządzenia o dużym zapotrzebowaniu na dane (kamery, telewizory, AGD)	Sterowanie bliskiego zasięgu (zamki, urządzenia ubieralne)	Oświetlenie, czujniki, termostaty
Zalety	Szybki, szeroko wspierany, nie wymaga huba	Wbudowany w telefony, bardzo niskie zużycie energii	Sieć mesh, niskie zużycie energii, obsługa 100+ urządzeń
Ograniczenia	Wysokie zużycie energii, nieoptymalny dla czujników	Krótki zasięg, ograniczone zastosowanie w automatyzacji	Wymaga huba, czasem problemy ze zgodnością
Typowe zastosowanie	Mieszkalne i komercyjne	Mieszkalne i komercyjne	Komercyjne i przemysłowe

Tabela 3.1. Porównanie: Wi-Fi, Bluetooth LE i Zigbee [4, 5, 6, 7, 8, 9, 10]

Właściwość	Z-Wave	Thread	Matter
Częstotliwość	Sub-GHz (868 MHz EU, 908 MHz US)	2.4 GHz	Thread / Wi-Fi / Ethernet
Zasięg	do 100 m na hop, >600 m w mesh (Sub-GHz, mała interferencja)	do 30 m na hop, >500 m w mesh (IPv6, niski narzut, mesh self-healing)	Zależny od fizycznej warstwy: Thread/Wi-Fi/Ethernet — nawet >1000 m (z routowaniem mesh lub siecią lokalną)
Główne zastosowania	Czujniki bezpieczeństwa, przełączniki, termostaty	Czujniki, zamki, oświetlenie	Uniwersalna kontrola inteligentnego domu
Zalety	Wysoka kompatybilność, dobry zasięg, niska interferencja	Sieć IP, samonaprawiająca się, niskie opóźnienia	Zunifikowany standard, działa z większością ekosystemów
Ograniczenia	Własnościowy standard, wymaga huba, zależny od regionu	Nowy standard, wymaga routera brzegowego	Wciąż rozwijany, ograniczona liczba urządzeń
Typowe zastosowanie	Mieszkalne	Mieszkalne (coraz częściej stosowane)	Mieszkalne

Tabela 3.2. Porównanie: Z-Wave, Thread i Matter [11, 12, 13, 14, 15]

Projektując uniwersalny sterownik istotną jest wysoka kompatybilność z dużą liczbą popularnych systemów. Jako iż Thread i Matter to stosunkowo nowe standardy, nie są one tak szeroko wspierane. Z pozostałych rozważanych protokołów, Bluetooth Low Energy najmniej wpasowuje się w rolę urządzenia Smart Home. Mimo bardzo dobrej wydajności elektrycznej, ograniczony zasięg jest problemem stawiającym go na niekorzystnej pozycji. Natomiast Wi-Fi jest wręcz w odwrotnej sytuacji: doskonały zasięg, okupiony wysokim zużyciem energii. Oba pozostałe protokoły Zigbee oraz Z-wave świetnie spisałyby się do komunikacji omawianego sterownika. Jednak to Zigbee wychodzi na prowadzenie, oferując sieć Mesh (opisaną w punkcie 3.2.3), która znacząco jest w stanie podbić zasięg sygnału, oraz będąc trochę bardziej powszechnie stosowanym w gotowych ekosystemach.

3.2.2. Zabezpieczenia oraz najczęstsze ataki

Kolejnym aspektem jest bezpieczeństwo i podatność na ataki. Zagrożenia przy skompromitowanej sieci, a także z urządzeń podłączonych do internetu zostały opisane w punkcie 2.4. Poniżej przedstawione zostały zagrożenia oraz zabezpieczenia poszczególnych protokołów. Źródła wykorzystane do stworzenia zestawienia:

- Bluetooth: [43, 44, 45, 46]
- Thread: [47, 48]
- Matter: [49, 50, 51]
- Z-Wave: [52, 53, 54]
- Zigbee: [55, 56, 57, 58]

3.2.2.1. Bluetooth LE

Zagrożenia

- **Podśluch w procesie parowania:** W przypadku korzystania z parowania “Just Works” nie jest wykorzystywane żadne szyfrowanie ani uwierzytelnienie. Atakujący znajdujący się w zasięgu sygnału może podejrzeć klucze komunikacji i wykorzystać je do odczytywania przyszłych wymian informacji.
- **Man-in-the-Middle (MITM):** Atak ten polega na przerwaniu procesu parowania w taki sposób, że pomiędzy oba urządzenia wchodzi kolejne, atakującego, udające

jednocześnie obie strony. Sprzęty ofiary myślą, że komunikują się bezpośrednio ze sobą, a tak naprawdę ich komunikaty są przetwarzane i podawane dalej przez hakera.

- **Spoofing adresu MAC:** Spoofing (podszywanie się) pozwala na udawanie urządzenia, ponieważ adresy MAC w niektórych systemach Bluetooth się nie zmieniają. Znając ten adres można łatwo oszukać drugą stronę, z którą już było nawiązywane wcześniej połączenie.
- **SweynTooth:** SweynTooth to lista znanych luk w zabezpieczeniach protokołu. Starsze urządzenia bez wprowadzonych aktualizacji zwalczających są wciąż podatne na ataki wykorzystujące te słabości.
- **Atak KNOB:** Bluetooth jest podatny na ten atak, który w skrócie wymusza sposób parowania, umożliwiając masowe zgadywanie klucza (Brute Force).
- **Nie wymagane szyfrowanie:** Szyfrowanie jest zbędne do działania protokołu, przez co wiele urządzeń, w tym niektóre inteligentne zamki komunikując się wysyłają hasła jako zwykły tekst. Każdy w zasięgu sygnału może go podsłuchać i bez problemów przeczytać jego zawartość.
- **Zakłócenia sygnału:** Protokół działa w paśmie 2,4 GHz, przez co jest podatny na wszelkiego rodzaju zakłócenia. Atakujący może wywołać silny sygnał przerywający komunikację urządzeń. Dodatkowo takie zakłócenia mogą się pojawić też z innych źródeł, takich jak np. mikrofalówka.

Zabezpieczenia

- + **Szyfrowanie AES-128 CCM:** Protokół umożliwia włączenie szyfrowania asynchronicznego, które jest bardzo czasochłonne do złamania.
- + **LE Secure Connections:** W wersji Bluetooth LE 4.2 wprowadzono listę ulepszonych zabezpieczeń, takich jak ECDH, które pozwala na bezpieczną wymianę kluczy.
- + **Losowy adres MAC:** Aby zwalczyć wcześniej wymieniony spoofing MAC, możliwe jest włączenie ich losowej generacji.
- + **Dodatkowe tryby parowania:** Numeric Comparison and Passkey Entry to dwa sposoby parowania urządzeń, które nie są najwygodniejsze, ale dostarczają ochronę przed atakami MITM.

- + **Niski zasięg:** To co może się wydawać przeszkodą w wygodzie użytkowania, jest dużą zaletą w bezpieczeństwie. Atakujący musi być fizycznie blisko urządzenia, aby podjąć próby włamania.

3.2.2.2. Thread

Zagrożenia

- **Jeden, wspólny klucz sieciowy:** Komunikacja wszystkich urządzeń w sieci jest szyfrowana tym samym kluczem. W przypadku jego wycieku, cała struktura jest skompromitowana.
- **Fałszywy commissioner:** Commissioner to pewny status urządzenia próbującego połączyć się z siecią. W przypadku zdobycia kodów PSKd, atakujący może dodawać własne, złośliwe węzły.
- **Ataki Replay:** Replay polega na przechwyceniu autoryzowanej wiadomości, i wysłaniu jej kopii do odbiorcy, który myśli, że otrzymał dwie od zaufanego nadawcy, co może prowadzić do manipulacji systemem (na przykład ponowne otwarcie drzwi). Bez poprawnie skonfigurowanych zabezpieczeń, system jest podatny na te ataki.
- **Atak na router graniczny:** Atakujący mogą dostać się do sieci poprzez niezwyfikowane węzły.
- **Zakłócenia sygnału:** Podobnie jak w przypadku Bluetootha Thread operuje na paśmie 2.4 GHz, które jest podatne na zakłócenia.

Zabezpieczenia

- + **Szyfrowanie AES-128 CCM:** Szyfrowanie jest standardem i nie jest potrzebna dodatkowa konfiguracja, aby je włączyć.
- + **DTLS:** Jest to rodzaj warstwy zabezpieczeń, używanej przy dołączaniu nowych urządzeń.
- + **Unikalne kody PSKd:** Każde urządzenie ma swój unikatowy kod, przez co bardzo utrudnione jest zgadnięcie go w celu podszycia się.
- + **Ochrony Nonce i Frame:** Oba zabezpieczenia bronią przed atakami Replay. Frame to numerowanie wysyłanych zapytań. Odbiornik przyjmuje tylko zapytania o

numerze większym, niż ostatnio odebrane. Nonce to unikalny dla każdej wiadomości numer, który utrudnia analizowanie szyfrogramu.

- + **Nowy protokół bez historii ataków:** Jako iż ten protokół jest jeszcze dość rzadko spotykany, nie był atakowany tak wiele razy jak inne. W rezultacie nie odkryto jeszcze żadnych skutecznych metod na atakowanie w środowisku produkcyjnym.

3.2.2.3. Matter

Zagrożenia

- **Spoofing urządzeń:** W przypadku wykradnięcia certyfikatu DAC, można się z dużym powodzeniem podszywać pod prawdziwe urządzenie.
- **Atak DoS podczas procesu handshake:** W momencie nawiązywania połączenia, szczególnie przy sprawdzaniu uwierzytelnienia, atakujący może wysłać dużą ilość komunikatów, co może przeciążyć odbiornik, doprowadzając do awarii urządzenia bądź błędu akceptującego fałszywe dane.
- **Wycieki metadanych:** Protokół udostępnia informacje o innych urządzeniach w sieci oraz ich rozkładzie. Jest to dużo ważnych informacji, które są łatwo dostępne po przejęciu kontroli w tylko jednym punkcie.
- **Możliwe błędy implementacji:** Architektura zabezpieczeń jest nowa i dobrze udokumentowana, ale pojawiają się pierwsze doniesienia o błędach implementacyjnych. Niektórzy producenci popełniają krytyczne błędy, takie jak nieszyfrowanie kluczy.

Zabezpieczenia

- + **Certyfikaty DAC, PAI oraz PAA:** Każde urządzenie ma własny certyfikat DAC, potwierdzony PAI, który z kolei jest potwierdzony PAA. Stworzenie fałszywego certyfikatu jest niemalże niemożliwe.
- + **SPAKE2:** Procedura wymiany kluczy uwierzytelniających, która pozwala na bezpieczną komunikację, nawet w przypadku bycia podsłuchiwanym.
- + **Szyfrowanie E2E (AES-128-GCM):** Szyfrowanie E2E (End to End), czyli pomiędzy pojedynczymi urządzeniami jest wysoce bezpieczne, nawet w przypadku komunikacji przez chmurę.

- + **ACL (Access Control List):** Specjalny system zapewniający szczegółową kontrolę nad pozwoleniami.
- + **OTA z podpisem:** Aktualizacje urządzeń muszą mieć certyfikowany podpis producenta. Bez niego żadne oprogramowanie nie zostanie zainstalowane.

3.2.2.4. Z-Wave

Zagrożenia

- **S0: klucz 00000000:** W starszych wersjach protokołu używano tego systemu: wszystkie urządzenia korzystały z tego samego statycznego klucza.
- **Atak Downgrade:** Podczas parowania możliwe jest wymuszenie powrotu urządzenia do wspomnianego wcześniej S0.
- **Atak DoS:** Urządzenia zasilane baterią są bardzo podatne na spam wiadomości, ponieważ mogą się szybko rozładować.
- **Możliwy brak OTA:** Starsze urządzenia nie mogą zostać zaktualizowane, by wyeliminować luki w zabezpieczeniach.
- **Ataki typu side-channel lub fałszywego dołączenia:** Wczesne wersje S2, często w modułach wbudowanych, posiadają znane luki, które mogą zostać wykorzystane.

Zabezpieczenia

- + **Nowy protokół S2:** Używana wymiana kluczy ECDH jest współczesna i bezpieczna. Każde urządzenie ma własny klucz.
- + **Szyfrowanie AES-128:** Całkowite szyfrowanie informacji.
- + **Whitelista:** Urządzenia nie znajdujące się na liście nie będą widziane jako godne zaufania.
- + **Podpisane aktualizacje:** System podobny jak w przypadku Matter — opcjonalny, ale możliwy do zaimplementowania.
- + **Zwiększona odporność na zakłócenia:** Operowanie w częstotliwościach mniejszych niż GHz uodparnia na zakłócenia.

3.2.2.5. Zigbee

Zagrożenia

- **Podśluch w procesie parowania:** Domyślne klucze są publiczne i mogą być wykorzystane podczas parowania.
- **Atak Replay:** Bez poprawnej implementacji zabezpieczeń Frame i MIC, ataki Replay są dużym ryzykiem.
- **Zakłócenia sygnału:** Podobnie do innych protokołów, pasmo 2.4 GHz jest podatne na zakłócenia.
- **Podszywanie się pod urządzenia:** Znając klucz sieciowy, łatwo jest się podszyć pod urządzenia.
- **Atak ZLL:** Wiele urządzeń oświetleniowych jest podatnych na wymuszenie ponownego połączenia z siecią przy pomocy kluczy atakującego.

Zabezpieczenia

- + **Szyfrowanie AES-128** Wszystkie wiadomości w sieci są szyfrowane.
- + **Trust Center:** Jeden moduł, zazwyczaj koordynator, zarządza wszystkimi kluczami.
- + **Zigbee 3.0: Install Code:** Kod parowania unikalny dla każdego urządzenia, co zwalcza podsłuchiwanie.
- + **Liczniki ramek i MIC:** Oferują bezpieczeństwo przed atakami Replay oraz manipulacją integralnością.
- + **Zasięg mesh:** Krótki zasięg (jak w Bluetooth), a dzięki układowi mesh możliwe jest zaprojektowanie zasięgu tylko w wymaganych miejscach.

3.2.2.6. Zestawienie wyników

Aby lepiej zobrazować różnice w omawianych protokołach, podane wcześniej informacje zebrano w tabelę 3.3. Pogrubiona linia oddziela sekcję zagrożeń od sekcji zabezpieczeń.

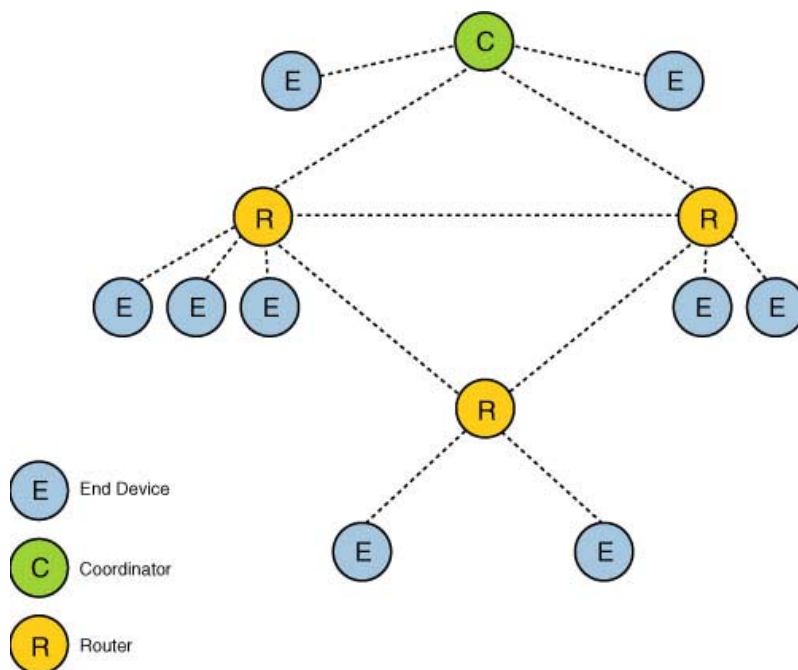
Właściwość	Wi-Fi	BLE	Zigbee	Z-Wave	Thread	Matter
Podśluch w procesie parowania	X	X	X			
Man-in-the-Middle (MITM)	X	X				
Spoofing adresu MAC / urządzenia	X	X	X			X
SweynTooth		X				
KNOB (Key Negotiation of Bluetooth)		X				
Replay (powtórzenie transmisji)	X		X		X	
Zakłócenia sygnału (Jamming)	X	X	X		X	X
Jeden wspólny klucz sieciowy			X		X	
Fałszywy commissioner					X	
DoS (atak przeciążeniowy)	X			X	X	X
Downgrade / wymuszenie starych kluczy	X			X		
Błędy implementacyjne / brak OTA	X			X		X
Szyfrowanie AES-128 (CCM / GCM)	X	X	X	X	X	X
LE Secure Conn / SPAKE2 / ECDH		X		X	X	X
Losowy MAC / unikalny PSKd / DAC		X	X	X	X	X
Trust center / certyfikaty			X		X	X
Numeracja ramek / MIC / Nonce			X	X	X	X
Whitelista / ACL / filtracja				X	X	X
OTA z podpisem				X	X	X
Zasięg lokalny jako ochrona	X	X	X	X	X	

Tabela 3.3. Zestawienie omówionych zagrożeń i zabezpieczeń rozważanych protokołów komunikacyjnych.

Zestawienie nie jest w stanie w pełni oddać bogatej listy właściwości każdego z protokołów. Nie wszystkie zagrożenia i mechanizmy zabezpieczeń mają jednakową wagę w praktycznych zastosowaniach. Wybór Zigbee nie wynika bezpośrednio z przewagi liczbowej, lecz z analizy całościowej, obejmującej także wcześniej omówione aspekty. Co istotne, każdy z omawianych protokołów może (przy odpowiedniej implementacji) stanowić podstawę do budowy bezpiecznego rozwiązania.

3.2.3. Topologia siatki (Mesh) w Zigbee

Oparcie komunikacji na sieci Mesh to naprawdę duży atut. Rysunek 3.1 przedstawia strukturę takiej sieci. Dzięki zastosowaniu routerów, można wyeliminować problem z brakiem zasięgu, a wiele ścieżek komunikacji pomiędzy samymi routerami czyni tę sieć wysoce niezawodną; w przypadku awarii jednego z routerów, możliwe jest przesyłanie danych przez inne, o ile sygnał jest wystarczająco silny.



Rysunek 3.1. Struktura sieci Mesh [3]

Ponieważ zadanie routera może być wysoce wymagające w kwestii energii, bardzo rzadko jest ta funkcjonalność spotykana w urządzeniach z zasilaniem bateryjnym. Jednak inne elementy, które są podpięte bezpośrednio do sieci, nie są limitowane.

Bardzo istotnym jest zaznaczenie, że funkcja routera jest opcją dodatkową, a nie głównym zadaniem wielu urządzeń. Dzięki temu możliwe jest wyeliminowanie potrzeby wielu wzmacniaczy sygnału, często w postaci nieestetycznego dongle'a zajmującego miejsce w gniazdku. Przykładowymi rozwiązaniami, które potrafią wzmacniać sygnał są często żarówki (IKEA LED2103G5, Philips 8719514491106), bądź też gniazdka czy przełączniki elektryczne (IKEA E160x/E170x/E190x, Sunricher SR-ZG9101SAC-HP-SWITCH-2CH).

Dla przykładu: Jednostka centralna ma za słaby sygnał, by wykryć czujnik ruchu na końcu długiego korytarza. Jednak ponieważ ten korytarz jest oświetlany inteligentnymi żarówkami z funkcją routera, sygnał jest przesyłany po nich. Jeśli jedna z żarówek się

przepali, a sygnał jest za słaby, by dotrzeć do następnej, może on podróżować przez inteligentne gniazdko, które znajduje się w miejscu, gdzie wypełnia przerwę. [59]

3.2.4. Kluczowe pojęcia protokołu Zigbee

Aby lepiej zrozumieć działanie urządzeń oraz zasady ich integracji z systemami Smart Home, warto zdefiniować kilka kluczowych terminów wykorzystywanych przez protokół Zigbee:

- **Endpoint:** Konkretny element komunikacji Smart Home. Służy do definicji rodzaju informacji, jakie są przesyłane, oraz funkcji, jakie można wykonać. Przykładowe endpointy to przełącznik światła ON OFF, Suwak wartości w zakresie, zmierzona temperatura. Zigbee umożliwia stosowanie wielu endpointów w jednym urządzeniu.
- **Klaster (ang. Cluster):** Zestaw powiązanych funkcji i danych reprezentujących jedną usługę, np. `OnOff`, `TemperatureMeasurement`, `LevelControl`. Klastry są przypisane do endpointów i mogą zawierać atrybuty oraz komendy (działania). Są swoistymi zestawami instrukcji do poprawnego działania endpointu.
- **Atrybut (ang. Attribute):** Pojedyncza wartość reprezentująca stan (np. aktualna temperatura) lub konfigurację (np. minimalny czas raportowania) czy siłę sygnału. Atrybuty są częścią klastrów. Jeden endpoint zazwyczaj posiada wiele atrybutów.
- **Zigbee End Device (ZED):** Urządzenie końcowe, zazwyczaj zasilane bateryjnie, które nie przekazuje danych dalej (nie jest routerem) i komunikuje się bezpośrednio z koordynatorem lub routerem. Charakteryzuje się niskim zużyciem energii. Urządzeniami końcowymi są np. inteligentne czujniki i termometry. Router do urządzenia końcowe z możliwością przesyłu danych dalej.
- **Trust Center:** Centralny element sieci Zigbee (zazwyczaj koordynator), odpowiedzialny za zarządzanie kluczami bezpieczeństwa i kontrolowanie procesu dołączania nowych urządzeń.
- **Install Code:** Unikalny kod przypisany do urządzenia Zigbee, wykorzystywany w procesie bezpiecznego parowania. Stanowi alternatywę dla domyślnych kluczy sieciowych i zwiększa poziom bezpieczeństwa.

3.3. Wybór płytki

Mając wybrany protokół, w jakim będzie nawiązywane połączenie, należało podjąć wybór dotyczący samej struktury urządzenia. Wykonanie sterownika możliwe jest na dwa sposoby: korzystając z gotowej płytki jako bazy, bądź tworząc własne urządzenie od podstaw. Obie możliwości wiążą się ze swoimi zaletami oraz wadami. W tej sekcji zostaną one dokładnie omówione.

3.3.1. Gotowe rozwiązanie

Korzystając z gotowego rozwiązania nie trzeba posiadać wiedzy o projektowaniu płytek oraz kompatybilności poszczególnych podzespołów. Takie urządzenia często posiadają również duże wsparcie producentów, oferując gotowe środowisko programistyczne oraz zbiór bibliotek do pracy z różnego rodzaju protokołami i modułami, wszystko w wybranym, znanym języku programowania. Szeroka oferta, zawierająca bardzo różne konfiguracje, umożliwia dobranie optymalnej płytki do potrzeb projektu. Wielu producentów oznacza dużą konkurencję, co w rezultacie skutkuje przystępnymi cenami za sprzęt o pożądanych parametrach.

Niestety to podejście ma również szereg wad, które podważają skuteczność tej metody. Przede wszystkim, mimo szerokiego wachlarza ofert, znalezienie rozwiązania które spełnia wymogi, ale ich nie przewyższa jest niemal niemożliwe. Producenci starając się przyciągnąć klientów, tworzą układy z jak największą liczbą funkcji. Na pierwszy rzut oka jest to bardzo korzystne i tak jest w większości przypadków, jednak próbując stworzyć moduł o bardzo konkretnym zapotrzebowaniu, staje się to problemem. Płytki wspierające wyłącznie Zigbee (bez Wi-Fi bądź Bluetooth) są rzadziej spotykane w ogólnodostępnej ofercie i często trudniejsze w implementacji (wymagają programatora/debuggera). Komponenty potrzebne do ich obsługi wliczane są w cenę, co przy założeniu produkcji masowej jest czynnikiem o dużym znaczeniu. Dodatkowo procesory o wysokiej wydajności potrzebne do obsługi tych protokołów nie tylko są droższe, ale również skutkują większym zużyciem energii, co także jest niepożądanym efektem ubocznym.

Duża ilość funkcjonalności często wiąże się również ze znacznym wzrostem wymiarów urządzenia. Ponieważ dodatkowe protokoły wymagają mocniejszych procesorów, dodatkowo udostępnia się użytkownikowi więcej pinów do korzystania z tej mocy, co wymaga wydłużenia samej płytki. Projektując sterownik uniwersalny, każdy milimetr ma duże

znaczenie, a nierzadko spotyka się rozwiązania nawet dwukrotnie krótsze od innych, choć przeważnie droższe.

Ostatnią kwestią jest omawiane już wcześniej bezpieczeństwo. Podobnie jak w przypadku całych gotowych inteligentnych modułów, same płytki też powinny być traktowane z dużą dozą ostrożności. Niestety niewykluczone jest, iż rozwiązanie, które ma wbudowaną obsługę Wi-Fi, będzie kontaktować się z serwerami producenta, wysyłając informacje o swoim działaniu. Użytkownik nie ma pełnej kontroli nad architekturą urządzenia, przez co może więc nie być świadomy obecności dodatkowych komponentów, takich jak moduły komunikacyjne.

Dodatkowo do programowania tego typu płytek najczęściej używa się połączenia USB. Możliwym jest, że przy podłączeniu dojdzie do instalacji złośliwego oprogramowania na komputerze użytkownika (BadUSB).

3.3.2. Własne rozwiązanie

Wykonując własną płytkę, można wyeliminować większość wad gotowych rozwiązań, tracąc przy tym część ich zalet.

Zdecydowanie największym atutem tego podejścia jest całkowita wolność przy projektowaniu możliwości i parametrów modułu. Tworząc cały układ od podstaw, możliwe jest wykorzystanie tylko niezbędnych komponentów o najniższych wymaganych specyfikacjach, optymalizując nie tylko cenę, ale także zużycie energii oraz wymiary urządzenia.

Osobiście wykonany projekt da pewność o zachowaniu prywatności, ze względu na znajomość użytych komponentów oraz wiedzy o całości układu. Dodatkowo eliminując zbędne elementy, takie jak antena Wi-Fi, fizycznie uniemożliwiane jest potencjalnym hakerom wykorzystanie ich przeciw właścicielowi.

Jednak ogromne zalety tego podejścia są nieoddzielne od równie dużych problemów, jakie wiążą się z jego implementacją. Prawdopodobnie największym z nich jest wymóg posiadania wiedzy o projektowaniu takich układów. Takie przedsięwzięcie nie jest łatwe, co zdecydowanie zmniejsza grupę osób, które mogłyby mu podołać.

Kolejną kwestią jest cena. Mimo zmniejszenia kosztów modułów potrzebnych do budowy płytki, ich zakup jest często niemożliwy w małych ilościach. Często jedynymi opcjami

są zakup hurtem (w tysiącach sztuk) bądź dużo wyższa cena za mniejsze ilości egzemplarzy.

Zakładając jednak dyspozycję niezbędnymi elementami, do ich montażu potrzebny jest specjalistyczny sprzęt, cenowo nieporównywalnie przekraczający koszt jednej płytki. Osoba nie posiadająca odpowiednich narzędzi bądź umiejętności, ma możliwość skorzystania z usług firm zajmujących się wykonywaniem pojedynczych urządzeń. Często oferują składniki po niższej cenie, ze względu na własny zakup w ilościach hurtowych. Wykonanie własnego projektu przy pomocy takich usług jest dość dużym wydatkiem.

Planując własnoręczne wykonanie płytki należy pamiętać o kosztach realizacji projektu. Jest ona wysoce nieopłacalna w przypadku tworzenia pojedynczych sztuk, jednak przy produkcji masowej oferuje bardzo duże oszczędności, co czyni korzystanie z gotowych rozwiązań niepraktycznym.

3.3.3. Przegląd gotowych rozwiązań

Analizując informacje zawarte w powyższych podpunktach, do realizacji projektu postanowiono skorzystać z gotowego rozwiązania. Decyzję poparto następującymi argumentami:

- **Charakter projektu:** Prototyp wykonany na rzecz tego dokumentu ma na celu jedynie wykazać możliwości realizacji jego założeń w stopniu nieoptymalnym. Nieistotnym jest osiągnięcie najlepszych wyników na tym etapie, a jedynie sprawdzenie teorii i wykonanie testów wskazujących na jej poprawność.
- **Wymagania czasowe:** Zaprojektowanie oraz utworzenie własnego układu może być wysoce czasochłonne. Dodatkowo w sytuacji braku ważnych komponentów lub błędnej konstrukcji, zaobserwowanych dopiero w trakcie testów, na gotowość następnej wersji sterownika poświęcony jest dodatkowy czas. Korzystając z gotowego rozwiązania posiada się pewną gwarancję działania układu.
- **Porównywalne parametry:** Własne rozwiązanie ma możliwość uzyskania znacznej przewagi w kontekście parametrów takich jak zużycie mocy, rozmiary, oraz cena. Jednak by ją uzyskać należy przejść przez wiele iteracji i prototypów wraz z ich testami i poprawkami w oprogramowaniu. Biorąc pod uwagę wcześniej wymienione ramy czasowe i charakter projektu, tworzenie własnego układu na tym etapie jest wysoce niepraktycznym.

- **Brak odpowiednich umiejętności:** W momencie realizacji projektu nie posiadano żadnego doświadczenia w produkcji takich układów, a biorąc pod uwagę wysoką złożoność wymaganego sterownika pod względem fizycznym jak i programowym, uznano, iż osiągnięcie odpowiednich umiejętności w tej dziedzinie wymagałoby zbyt dużo czasu.

Na rynku istnieje wiele rodzajów modułów, które można rozważyć jako podstawa sterownika w projekcie. Poniżej zostały przedstawione niektóre z rozważanych pozycji, z podziałem na odpowiednie kategorie.

3.3.3.1. Gotowe uniwersalne mikrokontrolery

To są płytki deweloperskie zawierające mikrokontroler możliwy do programowania przez użytkownika. Rozważane tu urządzenia zawierają wbudowane wsparcie różnych protokołów. Często jest bogate środowisko deweloperskie stworzone przez producenta, co pozwala na szybkie i proste korzystanie z możliwości produktu. Szeroko stosowane są interfejsy USB, dzięki czemu nie potrzebne są dodatkowe części by mieć dostęp do funkcji urządzenia.

Są to rozwiązania cechujące się jak największą prostotą dla korzystania przez użytkowników, przez co często posiadają ogromne społeczności pasjonatów, dzielących się swoimi projektami oraz napotkanymi problemami w celu ich rozwiązania. Produkty posiadają wiele wersji, pozwalając na trafniejsze dopasowanie do potrzeb klientów.

Poniżej przedstawiono przykłady modułów, które były rozważane jako podstawa projektu:

Arduino UNO

- + Środowisko i język przyjazny początkującym, pozwalający na proste i szybkie rozbudowywanie oprogramowania.
- + Ogromna społeczność, gwarantująca dużą ilość projektów, poradników i dokładnych dokumentacji.
- Brak natywnego wsparcia bezprzewodowych protokołów komunikacyjnych. Wymaga dodatkowych modułów i ich konfiguracji.
- Nisko wydajny 8-bitowy procesor, który nie wspiera wielowątkowości. Istnieje możliwość jej symulowania, lecz jest ona bardzo ograniczona i mocno spowalnia system.

- Ograniczona ilość pamięci RAM oraz brak systemu operacyjnego, co ogranicza możliwości rozbudowanych aplikacji.

Arduino Mini

- + Małe wymiary, co jest przydatne do produkcji urządzeń, gdzie liczy się rozmiar.
- + Tańszy od UNO, jednocześnie zachowując środowisko i społeczność.
- Brak interfejsu USB: wymaga osobnego programatora. Jednocześnie znacznie utrudnia debugowanie.
- Powiela wady UNO, z naciskiem na jeszcze niższą wydajność.

ESP32

- + Wbudowane Wi-Fi i Bluetooth: szeroka funkcjonalność komunikacyjna.
- + Duża moc obliczeniowa.
- + Niskie zużycie energii.
- + Środowisko ESP-IDF, pozwalające na niskopoziomowe zarządzanie komponentami.
- + Duża społeczność, oferująca gotowe projekty i poradniki.
- + Obsługa systemu czasu rzeczywistego (RTOS) i dwóch rdzeni pozwala na równoległe wykonywanie zadań.
- Brak wsparcia pozostałych rozważanych protokołów bez zewnętrznych modułów.
- Środowisko ESP-IDF wykorzystuje język C oraz niskopoziomowe podejście, co czyni je mniej przystępnym dla początkujących.

ESP32-C6

- + Wbudowane Zigbee, Thread, Wi-Fi i BLE, zapewniające szerokie wsparcie komunikacji.
- + To samo środowisko i część społeczności co wersja podstawowa.
- Stosunkowo nowy, przez co z uboższą dokumentacją i mniejszą ilością przykładów.

- Lekko wyższe zużycie energii niż w dedykowanych SoC Zigbee
- Nadal słabe wsparcie w środowisku Arduino – większość projektów opiera się na ESP-IDF, które jest mniej przyjazne.

ESP32-H2

- + Niskie zużycie energii przez optymalizację pod Zigbee i BLE.
- + Małe wymiary czyniące go idealnym do małych urządzeń końcowych (end device).
- Brak innych protokołów, np. Wi-Fi: ograniczone możliwości debugowania i OTA.
- Wczesna faza wsparcia, oferująca jeszcze mniej przykładów i gotowych bibliotek niż ESP32-C6.
- Bardzo ograniczona dostępność: trudniejszy do zdobycia niż inne wersje ESP32.

3.3.3.2. Dedykowane układy SoC

SoC: System on a Chip. To są rozwiązania specjalnie zaprojektowane pod wsparcie konkretnego protokołu, oraz często także urządzenia. Charakteryzują się brakiem portu USB, wymagając osobnego programatora, często w połączeniu z brakiem dokumentacji ani środowiska. Jest to efekt uboczny tego, iż są wysoce pożądane przez firmy, ze względu na ich wysoką wydajność i optymalne zużycie energii.

Silicon Labs EFR32MG21 / EFR32MG24

- + Zaprojektowane specjalnie do obsługi Zigbee, Thread oraz Bluetooth Low Energy.
- + Bardzo niskie zużycie energii, co czyni je idealnymi do urządzeń bateryjnych.
- + Wysoka jakość układów - stosowane w produktach IKEA, Philips Hue, itp.
- Brak interfejsu USB: wymagany zewnętrzny programator (J-Link).
- Często dostępne tylko jako część zamkniętego środowiska z ograniczoną dokumentacją.
- Wymagane środowisko Simplicity Studio, które jest mniej intuicyjne dla nowych użytkowników.

Texas Instruments CC2652P / CC2652R / CC2652RB

- + Wysoka stabilność i zasięg sygnału: układy z wzmacniaczem (wersja P) są idealne jako routery Zigbee.
- + Wspierane przez otwarte projekty, np. Zigbee2MQTT - duże wsparcie społeczności.
- + Niskie zużycie energii i wsparcie dla BLE oraz Thread.
- Brak Wi-Fi, brak USB - potrzebny host lub zewnętrzny MCU.
- Mimo wsparcia open-source, wymaga skomplikowanej konfiguracji narzędzi (Code Composer Studio, Flash Programmer).
- Dostępność jest ograniczona, przez co zakup jest problematyczny.

NXP JN5189 / JN5168

- + Obsługa Zigbee i BLE, w tym tryby niskiego poboru energii.
- + Czasem spotykane w produktach OEM, np. niektóre inteligentne gniazdka i piloty.
- Bardzo ograniczona dokumentacja i mała społeczność.
- Niska dostępność na rynku i wsparcie tylko w zamkniętych środowiskach NXP.
- Trudne do wykorzystania we własnym projekcie ze względu na nieintuicyjne narzędzia i brak przykładów.

3.3.3.3. Moduły konsumenckie/OEM ze wsparciem Zigbee

OEM: Original Equipment Manufacturer. To są zazwyczaj gotowe moduły, przeznaczone do stosowania jako część układu. Nie są projektowane z myślą o rozbudowywaniu czy przeprogramowywaniu. Istnieją możliwości by tego dokonać, jednak może się to wiązać z niespodziewanymi skutkami. Dzięki ich charakterystyce oferują niesamowicie niską cenę przy produkcji masowej.

Tuya TYZS3

- + Bardzo niska cena, szczególnie przy zamówieniach hurtowych.
- + Małe rozmiary, przez co łatwy do zintegrowania z innymi układami.

- Oprogramowanie fabryczne objęte vendor lock-in: konieczność modyfikacji by mieć pełną kontrolę.
- Brak oficjalnej dokumentacji, a większość wiedzy pochodzi z reverse engineeringu.
- Możliwe problemy ze stabilnością i aktualizacjami po flashowaniu.

Sonoff ZBBridge / ZBMini / Dongle-P

- + Gotowe moduły Zigbee dostępne na rynku konsumenckim, przez co łatwe do zdobycia.
- + Możliwość zmiany oprogramowania na Tasmota-Zigbee lub Zigbee2Tasmota.
- + Niska cena jak na urządzenia z gotową obudową i zasilaniem.
- Produkty projektowane jako końcowe, przez co trudne do modyfikacji fizycznie i programowo.
- Brak dokumentacji sprzętowej: często brak dostępu do pinów czy schematów.
- Ograniczone zastosowanie: często działają tylko jako koordynator lub przekaźnik, a nie jako pełnoprawny sterownik.

3.3.3.4. Platformy hybrydowe/mostowe

W tej kategorii znajdują się jednostki które są potężne w porównaniu z wcześniejszymi przykładami. Często posiadają własny system operacyjny, co czyni je małymi komputerami. Umożliwia to ogromną kontrolę i dogodny wybór środowiska i języka programowania. Podejście hybrydowe w tym kontekście odwołuje się do podpięcia zewnętrznego modułu, w przypadku braku natywnego wsparcia wybranego protokołu. Te urządzenia ze względu na swoją charakterystykę (Wysoka cena w zamian za dużą moc obliczeniową i wyższe zużycie energii) dużo lepiej sprawdzają się jako huby i jednostki centralne inteligentnych domów, niż urządzenia końcowe w większości przypadków.

Raspberry Pi (z modułem Zigbee, np. CC2531 / CC2652P)

- + Możliwość instalacji systemu operacyjnego (np. Linux) i dowolnego języka programowania.

- + Pełna kontrola nad środowiskiem: możliwość korzystania z logów, debugowania, sieci i automatyzacji.
- + Obsługa narzędzi takich jak Home Assistant, Zigbee2MQTT, Node-RED.
- + Szerokie wsparcie społeczności oferującej wiele poradników i projektów DIY.
- Wysokie zużycie energii wykluczające zastosowanie zasilania bateryjnego.
- Dużo wyższa cena w porównaniu do mikrokontrolerów.
- Duży rozmiar fizyczny, często wymagający dodatkowych akcesoriów (obudowa, chłodzenie, zasilacz).
- Brak natywnego wsparcia dla protokołów takich jak Zigbee - wymagany zewnętrzny dongle i jego konfiguracja.

Alternatywne SBC: Odroid, Orange Pi, Banana Pi

- + Wydajniejsze lub bardziej rozbudowane niż Raspberry Pi (np. więcej RAM, porty SATA).
- + Również obsługują Linux i popularne narzędzia automatyzacji.
- + Dobre do zastosowań serwerowych lub wielozadaniowych.
- Mniejsze wsparcie społeczności niż Raspberry Pi.
- Problemy ze zgodnością: nie wszystkie narzędzia (np. Zigbee2MQTT) wspierają je oficjalnie.
- Brak standaryzacji ze względu na różnice sprzętowe.

Home Assistant Yellow / Green

- + Zbudowane specjalnie jako centrum inteligentnego domu: gotowe rozwiązanie.
- + Wersja Yellow posiada natywne wsparcie Zigbee (EFR32MG21).
- + Wysoka stabilność i kompatybilność z Home Assistant.
- Zamknięta architektura: ograniczona elastyczność poza Home Assistant.
- Trudno dostępne i relatywnie drogie.
- Wersja Green wymaga osobnego modułu Zigbee.

Komputer stacjonarny / laptop z modulem Zigbee

- + Największa moc obliczeniowa – możliwość uruchomienia dowolnego oprogramowania bez ograniczeń.
- + Doskonały do testów, symulacji i środowisk deweloperskich.
- + Pełna obsługa narzędzi sieciowych, analitycznych i systemowych.
- Zbyt duże zużycie energii i koszt, by stosować jako urządzenie wbudowane.
- Brak natywnego Zigbee - konieczność użycia zewnętrznego dongla i konfiguracji.
- Mało mobilne - trudno zastosować w warunkach produkcyjnych jako część systemu Smart Home.

3.3.4. Podsumowanie

We wczesnych etapach projektów w większości przypadków najlepszym podejściem jest wykorzystanie gotowego rozwiązania do przetestowania podstawowych założeń i udowodnienia teorii. Po potwierdzeniu możliwości uzyskania oczekiwanych wyników, można przystąpić do projektowania całego układu od podstaw, co zapewni optymalizację parametrów takich jak cena, rozmiar urządzenia, pobór energii oraz funkcjonalność. Jednak by ten proces przyniósł pożądane efekty, należy posiadać odpowiednie narzędzia i umiejętności, oraz przygotować się na kosztowny i długi proces iterowania przez prototypy, lecz rezultujący w taniej produkcji masowej.

Z gotowych rozwiązań wiele jest przeznaczonych do użytku w niezmienionej formie, a te oferujące lepsze parametry pod konkretne protokoły są często skomplikowane w obsłudze.

Po analizie możliwych opcji, ze względu na charakterystykę projektu, do budowy prototypu sterownika zdecydowano się użyć uniwersalnego kontrolera, a konkretnie ESP32-C6, ze względu na wbudowane, natywne wsparcie protokołu Zigbee, który został wybrany jako docelowa technologia komunikacyjna po analizie dostępnych rozwiązań, oraz lepsze wsparcie niż ESP32-H2.

3.4. Środowisko programistyczne

Pracę z wybranym ESP32-C6 umożliwiają dwa główne środowiska:

- **Arduino:** Przyjazne środowisko zaprojektowane głównie do pracy z produktami wytwórcy, jednak poprzez wielką społeczność przystosowane również do obsługi innych modułów. Niestety, nie będąc oficjalnym środowiskiem, nie umożliwia pełni kontroli, przez co niemożliwym bądź mocno utrudnionym jest osiągnięcie całkowitej funkcjonalności urządzeń ESP.
- **Espressif ESP-IDF:** Oficjalne środowisko producenta, pozwalające na pełnię kontroli nad urządzeniem, kosztem większej złożoności i konieczności pracy w języku C, będącym często nieprzejrzystym i problematycznym dla osób z nim nieobeznanych.

Ze względu na to, iż Arduino nie pozwalało na dogłębną manipulację parametrami Zigbee modułu (takich jak konfiguracja klastrów, atrybutów czy procesu komisjonowania), do projektu prototypu wykorzystano Espressif ESP-IDF, które umożliwiło napisanie zestawu programów oraz ich wgrywanie na urządzenie. Praca z tym środowiskiem okazała się trudna, co zostało dokładniej opisane w punkcie 4.5.

Rozdział 4

Testy oraz wyniki

Ten rozdział skupia się na wdrożeniu teorii w praktykę oraz wykazaniu możliwości zaproponowanego rozwiązania. Zostały tu zawarte dane które udało się uzyskać, oraz propozycje przyszłych eksperymentów, jakie w znacznym stopniu przyczyniłyby się do rozwoju projektu. Omówione też zostały problemy napotkane podczas tworzenia prototypu oraz podjęte kroki w celu realizacji postawionych założeń.

4.1. Platforma testowa

W celu przetestowania sterownika zostało zasymulowane przykładowe środowisko, w jakim może się on znaleźć. Jako sprzęt koordynatora wykorzystano Raspberry Pi 4 Model B, z systemem operacyjnym Home Assistant. Ponieważ RP4 nie wspiera natywnie protokołu Zigbee, zainstalowano Zigbee 3.0 USB Dongle Plus: ZB Dongle-E firmy Sonoff. Ponieważ konfiguracja dongle'a okazała się problematyczna, w końcowej wersji użyto dodatków Mosquitto broker oraz Zigbee2MQTT, które umożliwiły korzystanie z urządzenia.

4.2. Przeprowadzone testy

Ta sekcja poświęcona jest testom jakie udało się przeprowadzić oraz osiągniętym результатам. Do wykonywania pomiarów napięcia i natężenia wykorzystano miernik uniwersalny (multimetr) Uni-T UT890D+. W treści pojawiają się wartości poboru mocy, które były obliczane ze wzoru:

$$P \text{ [W]} = U \text{ [V]} \times I \text{ [A]}$$

Testy były wykonywane przy pomocy płytki prototypowej (płytki stykowo-stykowej).

4.2.1. Test platformy testowej z wykorzystaniem gotowych urządzeń

Aby podczas tworzenia prototypu mieć pewność, iż potencjalne problemy leżą po jego stronie, zakupione zostały dwa gotowe inteligentne urządzenia:

- Żarówka LED EMOS GoSmart A65 E27
- Termometr bezprzewodowy EMOS GoSmart EGS0102

Następnie została podjęta próba ich podłączenia do platformy testowej, i wykonania operacji potwierdzających poprawną komunikację. Żarówka bezproblemowo nawiązała połączenie z systemem, oraz udostępniła opcje kontroli parametrów. Za pomocą panelu w Home Assistant możliwe było zmienianie kolorów oraz jasności żarówki, jak i jej całkowite włączanie i wyłączanie. Na podstawie tych obserwacji uznano, iż platforma testowa jest sprawna i nie zakłóci wyników testów sterownika.

Testowany termometr nawiązał połączenie z systemem, jednakże pomimo wielu prób i wersji konfiguracji oraz restartów urządzenia, nigdy nie udało się otrzymać informacji od modułu. Mimo poprawnego nawiązania komunikacji, nie uznano go za pracującego prawidłowo. Wywnioskowano, iż urządzenie jest wadliwe, bądź odmawia współpracy innym koordynatorem niż producenta, co jest praktycznym przykładem “vendor lock-in’u”.

4.2.2. Test przykładu HA__on__off__light

Celem testu było wgranie na płytkę gotowego przykładu z pakietu Espresiff esp-zigbee-sdk. Miał on wykazać zdolność operacyjną urządzenia oraz poprawne skonfigurowanie środowiska.

Spośród wielu dostępnych przykładów wybrano HA__on__off__light, ze względu na prostą konstrukcję i widoczne efekty. Płytką miała zostać wykryta przez platformę testową, umożliwiając jej kontrolowanie diody na GPIO 15, włączając ją i wyłączając wyłącznie za pomocą środowiska Home Assistant.

Test wykonano pomyślnie.

4.2.3. Test trzech urządzeń

Celem testu było sprawdzenie możliwości kontroli trzech indywidualnych urządzeń podłączonych do jednego, wspólnego sterownika uniwersalnego. Ta funkcjonalność jest

kluczowa dla szerokiego zakresu zastosowań, który jest jego założeniem.

Kod z testu 4.2.2 został rozbudowany, tak by zgłaszane zostały 3 indywidualne endpointy. Listing 4.1 przedstawia sposób implementacji [60]. Przykład odbioru poleceń i różnicowania ich w kontekście wielu urządzeń jest przedstawiony na listingu 4.4.

```
static void esp_zb_task(void *pvParameters)
{
    /* initialize Zigbee stack */
    esp_zb_cfg_t zb_nwk_cfg = ESP_ZB_ZED_CONFIG();
    esp_zb_init(&zb_nwk_cfg);

    esp_zb_on_off_light_cfg_t switch_cfg = ESP_ZB_DEFAULT_ON_OFF_LIGHT_CONFIG();
    esp_zb_ep_list_t *ep_list = esp_zb_ep_list_create();
    for (int i = 0; i < 3; i++) {
        esp_zb_endpoint_config_t endpoint_config = {
            .endpoint = 1 + i,
            .app_profile_id = ESP_ZB_AF_HA_PROFILE_ID,
            .app_device_id = ESP_ZB_HA_ON_OFF_LIGHT_DEVICE_ID,
            .app_device_version = 0,
        };
        zcl_basic_manufacturer_info_t info = {
            .manufacturer_name = "CustomNameTest",
            .model_identifier = "LED_example",
        };
        esp_zb_ep_list_add_ep(ep_list, esp_zb_on_off_light_clusters_create(&switch_cfg), endpoint_config);
        esp_zcl_utility_add_ep_basic_manufacturer_info(ep_list, HA_ESP_LIGHT_ENDPOINT + i, &info);
    }
    esp_zb_device_register(ep_list);

    //esp_zb_device_register(esp_zb_on_off_light_ep);
    esp_zb_core_action_handler_register(zb_action_handler);
    esp_zb_set_primary_network_channel_set(ESP_ZB_PRIMARY_CHANNEL_MASK);
    ESP_ERROR_CHECK(esp_zb_start(false));
    esp_zb_stack_main_loop();
}
```

Rysunek 4.1. Funkcja `esp_zb_task` zmodyfikowana do deklaracji trzech endpointów

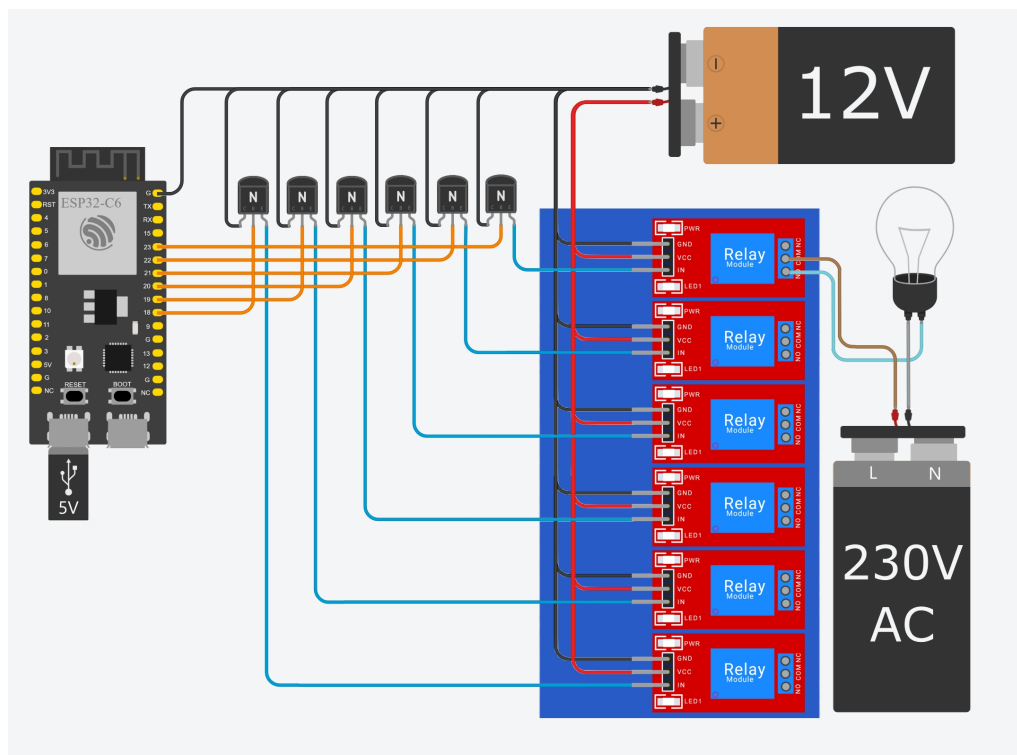
Po wgraniu zmodyfikowanego kodu na sterownik i odświeżeniu urządzenia w Home Assistant, ukazały się 3 osobne przełączniki, które pozwalały na kontrolę trzech oddzielnych diód (GPIO 20 - 22) w sposób indywidualny.

Wnioski: Test wykazał możliwość deklaracji wielu modułów podłączonych do tego samego sterownika oraz ich indywidualną, niezależną kontrolę przez Home Assistant.

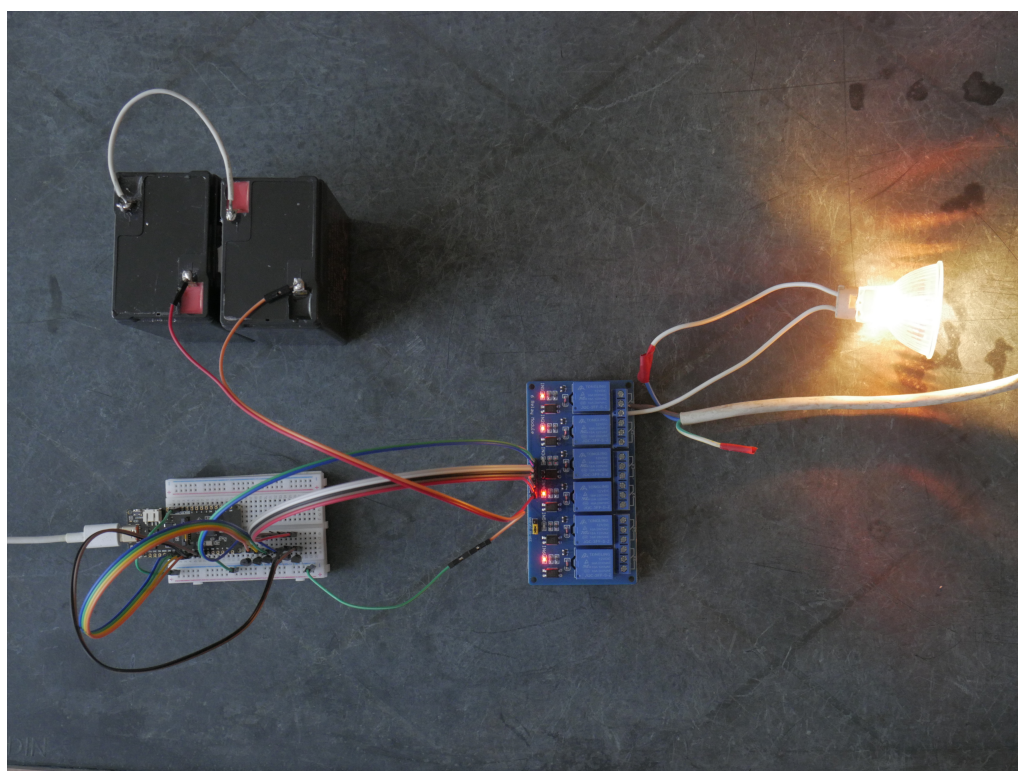
4.2.4. Test przekaźników

Celem testu było wykazanie możliwości kontroli zestawu przekaźników przy pomocy sterownika uniwersalnego oraz zmierzenie jego poboru mocy.

Rysunek 4.2 ilustruje diagram testowanego układu, którego fizyczne wykonanie jest widoczne na rysunku 4.3.



Rysunek 4.2. Schemat układu zastosowanego w teście przekaźników.



Rysunek 4.3. Zdjęcie faktycznego układu zastosowanego w teście przekaźników.

W teście wykorzystano gotowy moduł zawierający 6 przekaźników TONGLING JQC-3FF-S-Z. Do jego pracy użyto oddzielnego zasilania 12V DC. Aktywacja przekaźników następowała poprzez połączeniu pinów IN1 - IN6 z masą za pomocą tranzystorów kontrolowanych przez sterownik. Aktywowane przekaźniki wydawały słyszalny odgłos, oraz zapalały diodę kontrolną. Do wykazania poprawnej pracy układu do przekaźnika 1 podpięta została żarówka zasilana bezpośrednio prądem zmiennym 230V.

Modyfikując kod z testu 4.2.3 do Home Assistanta zostało zgłoszonych 6 endpointów. Każdy z nich bezpośrednio kontrolował odpowiedni tranzystor, przełączając napięcie na pinach GPIO 18 - 23. Listing 4.4 zawiera fragment zarządzający logiką sterowania odpowiednimi pinami GPIO. Nieporządek w numerowaniu wynika z poprawy różnicy adresowania endpointów przez Home Assistanta.

```
bool light_state = *(bool *)message->attribute.data.value;
uint8_t ep = message->info.dst_endpoint;

ESP_LOGI(TAG, "Endpoint %d: Relay set to %s", ep, light_state ? "On" : "Off");

switch (ep) {
    case 6:
        gpio_set_level(GPIO_NUM_18, light_state);
        ESP_LOGI(TAG, "GPIO 18");
        break;
    case 4:
        gpio_set_level(GPIO_NUM_19, light_state);
        ESP_LOGI(TAG, "GPIO 19");
        break;
    case 1:
        gpio_set_level(GPIO_NUM_20, light_state);
        ESP_LOGI(TAG, "GPIO 20");
        break;
    case 2:
        gpio_set_level(GPIO_NUM_21, light_state);
        ESP_LOGI(TAG, "GPIO 21");
        break;
    case 5:
        gpio_set_level(GPIO_NUM_22, light_state);
        ESP_LOGI(TAG, "GPIO 22");
        break;
    case 3:
        gpio_set_level(GPIO_NUM_23, light_state);
        ESP_LOGI(TAG, "GPIO 23");
        break;
    default:
        ESP_LOGW(TAG, "Unhandled endpoint: %d", ep);
        break;
}
```

Rysunek 4.4. Fragment funkcji `zb_attribute_handler` w teście przekaźników.

Po złożeniu układu wykonano testy sprawdzające niezależną kontrolę nad poszczególnymi przekaźnikami. Home Assistant wyświetlał 6 osobnych przełączników w jednym urządzeniu, a każdy z nich bezpośrednio aktywował i dezaktywował pojedynczy przekaźnik. Czas reakcji przekaźnika na kliknięcie przełącznika w kliencie HA był natychmiastowy, bez zauważalnych opóźnień.

Aby zmierzyć pobór mocy sterownika podczas jego operacji, do kabla zasilającego USB-C podpięto szeregowo multimetr. Zmierzono również napięcie zasilania, które wyniosło 5V. Na podstawie tych danych dokonano obliczeń poboru mocy przez sterownik w różnych etapach pracy. Uzyskane wyniki przedstawiono w tabeli 4.1. Widoczne liczby nie zawierają w sobie mocy zużywanej przez żarówkę, ani przez moduł przekaźników.

Stan pracy sterownika	Natężenie na wejściu (mA)	Pobierana moc (W)
Spoczynek: 0 przekaźników uruchomionych	82.6	0.413
Praca: 1 przekaźnik uruchomiony	127.2	0.636
Praca: 3 przekaźniki uruchomione	195.7	0.979
Praca: 6 przekaźników uruchomionych	320	1.6

Tabela 4.1. Zestawienie zużycia mocy przez sterownik uniwersalny w teście przekaźników.

Z danych uzyskanych z testów wynika, iż sterownik ma statyczne zużycie mocy w wysokości 0.413W, oraz średnia moc potrzebna na wysłanie sygnału przełączenia do przekaźnika wynosi w przybliżeniu 0.2W.

Wnioski: Z przeprowadzonego testu wynika, iż możliwe jest indywidualne kontrolowanie sześciu przekaźników w czasie rzeczywistym przy wykorzystaniu sterownika uniwersalnego, jednak pobór mocy potrzebny na jego operowanie jest stosunkowo wysoki.

4.2.5. Test przełącznika

Celem testu było uzyskanie kontroli nad innym komponentem w systemie Home Assistant, z wykorzystaniem fizycznego przełącznika podpiętego do uniwersalnego sterownika.

Do przeprowadzenia testu wykorzystano gotowy przykład `HA_on_off_switch`. Kod zmodyfikowano, tak aby sygnał został czerpany z pinu GPIO 9, który przez przełącznik

został podłączony do masy. Przesławianie przełącznika odpowiednio udrażniało i blokowało połączenie, co było wykrywane przez sterownik, wysyłający zmienne sygnały.

Po przeprowadzeniu rekonfiguracji sterownika w Home Assistant, możliwe było przesyłanie sygnałów. Wykorzystując wcześniej zaimplementowaną do systemu żarówkę (test 4.2.1) utworzono połączenie pomiędzy nią a sterownikiem w środowisku HA. Dzięki temu poprzez przełączanie stanów fizycznego przełącznika możliwe było włączanie i wyłączanie żarówki.

Wnioski: Sterownik pozwala na wykorzystanie fizycznego komponentu do kontroli innego urządzenia w sieci Smart Home.

4.2.6. Test potencjometru

Celem testu było uzyskanie większej kontroli nad innym komponentem w systemie Home Assistant, z wykorzystaniem fizycznego potencjometru podpiętego do uniwersalnego sterownika. Zmieniając położenie gałki potencjometru system powinien otrzymywać odpowiedni sygnał w zakresie 0-100, co umożliwiłoby dokładniejszą kontrolę nad komponentami.

Do przeprowadzenia testu wykorzystano potencjometr B5K 50° R1710N, podłączony do pinu GPIO 5 sterownika. Korzystanie z gotowego przykładu `HA_color_dimmable_switch` okazało się problematyczne, zmodyfikowano więc przykład `HA_temperature_sensor`. W ten sposób możliwe zostało stworzenie oprogramowania urządzenia typu EndPoint wysyłającego gotowy sygnał.

Rozwiązanie nie jest idealne, ponieważ Home Assistant nadal widzi sterownik jako czujnik temperatury, a uzyskiwany sygnał jest podawany w stopniach Celsjusza, jednak możliwym było napisanie własnego skryptu automatyzującego, powiązującego wysokość "temperatury" z jasnością zaimplementowanej żarówki (test 4.2.1). W rezultacie zmieniając położenie gałki potencjometru, żarówka świeciła jaśniej, bądź ciemniej.

Wnioski: Możliwe jest utworzenie połączenia z Home Assistant, przesyłając dokładny sygnał w zakresie 0-100, który można wykorzystać w automatyzacji. Gotowy przykład takiego urządzenia jest problematyczny w implementacji, lecz sposobem na obejście problemów jest podszycie się pod czujnik temperatury.

4.2.7. Test zasilania z baterii

Celem testu było wykazanie możliwości zasilania sterownika przy pomocy baterii oraz porównanie zużycia mocy w kontraście do energii dostarczanej przez zasilacz wykorzystując USB-C.

Do testu wykorzystano 3 baterie AA o napięciu 1.3V połączone szeregowo, dając łączne napięcie 3.847V. Układ podłączono poprzez multimetr do sterownika z wgranym przykładem `HA_on_off_light`, a następnie dokonano pomiarów przy przełączaniu diody. Następnie pomiary powtórzono zamieniając zasilanie z baterii na prąd z zasilacza o napięciu 4.957V. Dane z pomiarów zostały przedstawione w tabeli 4.2. Stan “przejście” oznacza najwyższy zanotowany skok w natężeniu podczas przełączania diody, spowodowane odebraniem sygnału i przetwarzaniem go.

Zasilanie	Stan	Natężenie na wejściu (mA)	Średnia pobierana moc (W)
Baterie	Dioda OFF	77.6 - 78.1	0.299
Baterie	Dioda ON	78.2 - 78.5	0.301
Baterie	Przejście	100	0.385
Zasilacz	Dioda OFF	80.9 - 81.3	0.402
Zasilacz	Dioda ON	81.5 - 82.2	0.406
Zasilacz	Przejście	105	0.520

Tabela 4.2. Zestawienie zużycia mocy przez sterownik uniwersalny w teście zasilania z baterii.

Wnioski: Sterownik pracuje normalnie przy zasilaniu z baterii. Zaobserwowano, iż zużycie mocy jest średnio niższe o 25% przy zasilaniu z baterii niż przy zasilaniu z zasilacza. Różnicę można wytłumaczyć logiką urządzenia opartą o 3.3V. Napięcie dostarczane musi zostać obniżone, co nie jest możliwe bezstratnie. Ponieważ baterie miały dużo bliższe napięcie do porządanego, uzyskały wydajniejsze zużycie mocy niż zasilacz. Tą teorię potwierdza fakt, iż 75% z 4.947V (napięcie zasilacza) wynosi 3.718V. Jest to wynik różniący się od napięcia baterii o zaledwie 0.13V.

Test wykazał więc, iż odpowiednie dopasowanie zasilania do potrzeb urządzenia może znacząco zmniejszyć pobór mocy.

4.2.8. Test dwóch czujników

Celem testu było wykazanie możliwości uzyskiwania informacji z wielu czujników wpiętych do sterownika uniwersalnego jednocześnie. W ten sposób udowodnione zostaną: możliwość pobierania informacji od sterownika przez jednostkę dowodzącą, oraz zdolność wykorzystania kilku różnych czujników w jednym urządzeniu.

Do wykonania testu użyto:

- Czujnik natężenia światła BH1750 GY-302
- Czujnik ciśnienia, wilgotności oraz temperatury BME280 5V

Czujniki zostały podłączone do sterownika za pomocą protokołu I2C. Ze względu na niestabilne fizyczne połączenie czujników w układzie, oprogramowanie zostało wyposażone w funkcje mające zapobiegać błędom systemu przy niewłaściwej inicjalizacji, bądź przy nieudanym wczytaniu danych z sensorów.

Po ponownym połączeniu sterownika z Home Assistant, widoczne były dane odnośnie temperatury oraz naświetlenia, zgodne z informacjami podawanymi bezpośrednio przez moduł w konsoli. Problematicznym okazał się przesył informacji o wilgotności, który na platformie testowej wynosił 0. Wywnioskowano, iż sposób prezentacji tej właśnie zmiennej jest diametralnie różny w Home Assistant oraz Espressif. Dane odnośnie ciśnienia nie posiadały własnej logiki w systemie ESP. By je przekazać, należałoby napisać własny cluster, którego stworzenie jest czasochłonne i problematyczne.

Wnioski: Sterownik umożliwia korzystanie z wielu czujników naraz, co potwierdzają dane o temperaturze i naświetleniu, będące zbierane przez dwa osobne sensory. Niektóre informacje są problematyczne do przesyłania w sposób natywny, co wykazuje dalsze niedogodności z korzystania z rozwiązania ESP.

4.2.9. Test stacji czujników z wyświetlaczem

Celem testu było wykazanie możliwości tworzenia endpointów do przesyłania informacji w obie strony z wykorzystaniem jednego sterownika uniwersalnego. Dowodem na taką właściwość było rozszerzenie układu z testu 4.2.8 o trzy-cyfrowy wyświetlacz siedmio-segmentowy. Ma on za zadanie pokazywania bieżącej temperatury, a poprzez Home Assistant, użytkownik ma mieć możliwość jego włączenia i wyłączenia.

Do wykonania testu wykorzystano wspomniany układ oraz wyświetlacz 5631AS, podłączony przy pomocy tranzystorów NPN oraz rezystorów 150Ω. Podczas testu zaobserwowano słabe podświetlenie segmentów G, które najprawdopodobniej wynikało z niesprawnego połączenia, jednak nie wpływało negatywnie na wyniki.

Podczas przeprowadzania testu zauważone zostały liczne anomalie:

- **Błędy oprogramowania przy problemach z odczytem danych z sensorów:**

Mimo wykorzystania sprawdzonego wcześniej kodu z poprzedniego testu jako podstawy, oraz dodania jedynie logiki wyświetlacza, sterownik zawieszał się, gdy napotykał problemy z kontaktem z czujnikami. Debuggując kod zauważono, że polecenie `vTaskDelay(pdMS_TO_TICKS(1000));`, które tworzy opóźnienie jednej sekundy, całkowicie zawieszało program, przez co nie zostało wykonane żadne inne polecenie. Błąd jest o tyle dziwny, iż linijka, na której się zatrzymywał program, nie znajduje się bezpośrednio w funkcji służącej do komunikacji I2C. Przy uruchamianiu sterownika z oboma czujnikami całkowicie odłączonymi program się nie zawieszał, poprawnie informując o braku danych.

- **Błędy w konfiguracji wstecznej:** Po napotkaniu wcześniejszego problemu powrócono do przetestowanych wcześniej wersji programu, zarówno z komunikacją Zigbee, jak i samym lokalnym zarządzaniem czujnikami. Programy które wcześniej działały i nie zwróciły błędów zarówno przy normalnej pracy, jak i przy słabym kontakcie z czujnikami, zaczęły się zachowywać w podobny sposób do wyżej opisanego.

- **Nieprawidłowe uruchamianie z wykorzystaniem baterii:** W wersji finalnej programu, podczas gdy sterownik był zasilany przez połączenie USB-C 4.795V, zachowywał się przewidywalnie i prawidłowo. Jednak w momencie zastosowania zasilania z baterii 3.9V (test 4.2.7) licznik utykał na liczbie 77.4, a sterownik resetował się w odstępie około jednosekundowym. Nie udało się doprowadzić do normalnej pracy układu.

- **Nieprawidłowe uruchamianie przy połączeniu przez multimetr:** W czasie pomiarów natężenia zauważono, iż sterownik zachowuje się inaczej w zależności od ustawień multimetra. Podczas gdy pomiar był w ustawieniu max 20A, układ pracował normalnie, a wynik maksymalny był na wysokości 0.14A. Jednak przy próbie uzyskania dokładniejszego wyniku, zmieniając maksymalną wartość na 600mA (0.6A, czyli o ponad czterokrotnie większą od maksymalnej notowanej) sterownik nigdy nie osiągnął normalnej pracy: resetował się w odstępach ponadsekundowych,

a wyświetlacz ponownie utknął na liczbie 77.4. Uznano, iż takie zachowanie może być spowodowane bardzo szybkim skokiem natężenia przekraczającym próg 600mA, sprawiającym problemy przy starcie płytki, jednocześnie będącym zbyt krótkim by zostać uchwyconym przez multimetr. Najwyższy wynik zaobserwowany wynosił 174.9mA.

W związku z zaistniałymi problemami, dane z czujników postanowiono symulować: zamiast czytywać prawdziwe liczby, parametry otrzymywały losową wartość w zakresie 0 - 99. Jako iż możliwość faktycznego czytywania została już wcześniej potwierdzona, to podejście miało wpływ jedynie na mierzone zużycie mocy (czujniki nie były podłączone, przez co nie pobierały energii).

Kod z testu 4.2.8 rozszerzono o funkcjonalność wypisywania temperatury na podłączonym wyświetlaczu, z dokładnością jednej dziesiątej. W połączeniu Zigbee zadeklarowano dwa różne endpointy, co w rezultacie sprawiło, że pod spisem danych z czujników w Home Assistant, pojawił się przełącznik. Dzięki zastosowanej logice, możliwe było kontrolowanie nim włączania i wyłączania wyświetlacza. Podczas testowania zauważono, iż pierwsze wyłączanie pozostawiało kilka segmentów lewej cyfry włączonych, lecz żadne następne tego nie ponawiały.

W czasie testu wykonano pomiary natężenia przy włączonym i wyłączonym wyświetlaczu. Wyniki pomiarów zostały zaprezentowane w tabeli 4.3.

Zasilanie	Stan	Natężenie na wejściu (mA)	Średnia pobierana moc (W)
Zasilacz	Wyświetlacz OFF	90	0.432
Zasilacz	Wyświetlacz ON	140	0.671
Baterie	Wyświetlacz OFF	-	-
Baterie	Wyświetlacz ON	80	0.308

Tabela 4.3. Zestawienie zużycia mocy przez sterownik uniwersalny w teście stacji czujników z wyświetlaczem.

Wnioski: Możliwa jest implementacja obu z testowanych wcześniej wersji endpointów w jednym sterowniku uniwersalnym, co daje komunikację w obie strony. Mimo tego, praktyczne wykonanie rozwiązania jest bardzo problematyczne. Stosowane środowisko programistyczne wytwarza niespodziewane błędy, a sposób prezentacji danych takich jak wilgotność i ciśnienie jest niekompatybilny z platformą testową.

4.2.10. Test maksymalnego zasięgu

Celem testu było znalezienie maksymalnej odległości, na jaką komunikacja między sterownikiem a platformą testową była zadowalająca: utrzymana bez większych opóźnień i komplikacji.

Test przeprowadzono przy pomocy drugiej osoby (pan Kacper Homeniuk), pełniącej rolę obserwatora. Eksperyment został wykonany wieczorem przy bezwietrznej pogodzie i w temperaturze ok. 25°C. Jako moduł sterownika wykorzystano układ zasilany bateriami z testu 4.2.7.

Platforma testowa została umieszczona na balkonie mieszkania. Zarządzający nieustannie, w różnych odstępach czasu przełączał ustawienie światła w Home Assistant. Obserwator znajdujący się na zewnątrz budynku w linii prostej oddalał się od platformy testowej, przez połączenie głosowe z zarządzającym potwierdzając działanie sterownika, który odpowiednio zapalał oraz gasił diodę LED.

Mimo iż z założenia maksymalny zasięg był przewidywany na odległość ok. 20 metrów, udało się utrzymywać nieprzerwane połączenie do odległości 70 metrów. Powyżej tej wartości komunikacja okazała się zbyt niestabilna, by uznać ją za zadowalającą. Home Assistant na takiej odległości zgłaszał siłę sygnału w wachaniach 50 - 80. Dla porównania, przy odległości 3 metrów ta wartość wynosiła ok. 255. Wartym uwagi jest obserwacja, iż po zresetowaniu zasilania sterownika, nie nawiązał on ponownego połączenia z platformą, do momentu zmniejszenia dystansu do ok. 10 metrów.

Ze względu na zaskakujące wyniki testu, do obliczenia odległości użyto koordynatów pobranych z serwisu Google Maps. Wynik poparto przybliżonymi obserwacjami otoczenia. Ze względu na prywatność, właściwe koordynaty uzyskane w teście nie zostaną upublicznione.

Wnioski: Nawiązana komunikacja protokołu Zigbee wykazała się znacznie wyższym zasięgiem niż przewidywano. Możliwymi zmiennymi wpływającymi na wynik testu mogą być nieokluczyna droga pomiędzy sterownikiem oraz platformą testową, oraz umiejscowienie platformy: stalowa rama balkonu mogła przyczynić się do spotęgowania sygnału, działając jako większa antena. Jednak mimo tego wyniku, po utracie połączenia, ponowne nawiązanie go okazało się nieskuteczne, aż do znacznego zmniejszenia dystansu pomiędzy sterownikiem a jednostką główną.

4.2.11. Test zużycia energii przez telefon

Celem testu było uzyskanie przykładowego zużycia energii przez znajome urządzenie, tak by uzyskać punkt odniesienia dla liczb deklarowanych przez urządzenia. Jeżeli czytelnik nie jest zaznajomiony z pojęciami dotyczącymi energii elektrycznej, ten test rzuca na nie światło i umożliwia ocenę wysokości zużycia energii przez mniej skomplikowane urządzenia.

Test polegał na pozostawieniu telefonu w trybie gotowości przez określony czas, a następnie porównaniu stanu baterii. Z zebranych danych możliwe jest wyliczenie zużycia energii.

Dane testowe:

- **Testowane urządzenie:** Smartphone Xiaomi Redmi Note 8 Pro (Stan: codzienny użytek przez 5 lat).
- **Uruchomione protokoły:** Wi-fi, Bluetooth, NFC, Lokalizacja. Aplikacje działające w tle: budzik, stoper.
- **Pojemność baterii:** 4500 mAh
- **Średnie napięcie baterii:** 3,9 V
- **Czas trwania testu:** 8 h 50 min = 8,83 h
- **Spadek poziomu baterii:** 81% → 76% (przyjęto 6% zużycia, uwzględniając margines błędu pomiaru)
- **Temperatura otoczenia:** 15 – 20°C

Obliczenia:

1. Zużyta pojemność:

$$Q = 0,06 \cdot 4500 \text{ mAh} = 270 \text{ mAh} = 0,27 \text{ Ah}$$

2. Średnie natężenie prądu:

$$I = \frac{Q}{t} = \frac{0,27 \text{ Ah}}{8,83 \text{ h}} \approx 0,0306 \text{ A}$$

3. Obliczenie mocy:

$$P = U \cdot I = 3,9 \text{ V} \cdot 0,0306 \text{ A} \approx 0,119 \text{ W}$$

Wnioski: Telefon pozostawiony w trybie czuwania z włączonymi dodatkowymi zbędnymi protokołami ma średni pobór mocy w wysokości **0,12W**.

4.3. Propozycje testów

Ta sekcja została poświęcona testom, których nie udało się zrealizować z różnych względów, jednak są one kluczowe do pełnego podglądu na możliwości przedstawionego rozwiązania. W przypadku badania innego urządzenia, w połączeniu z wyżej opisanymi testami w pełni wykażą parametry, które jak wynika z analizy rynku, są najważniejsze.

4.3.1. Test kompatybilności

Uniwersalność sterownika oznacza jego zdolność do pracy z każdym systemem. Aby udowodnić jego możliwości należałoby gotowe rozwiązanie przetestować pod względem nawiązywania połączenia i pełnej obsługi w jak największej liczbie dostępnych środowisk, takich jak:

- **Chmurowe i hybrydowe systemy Smart Home:** Tuya Smart, Google Home, Amazon Alexa, Apple Home (HomeKit), Samsung SmartThings, Philips Hue, Bosch Smart Home, Govee Home, Aqara Home, Meross, Kasa Smart, eWeLink (Sonoff), Tapo, SwitchBot, Shelly Cloud, Wiz, Xiaomi Home (Mi Home), Yeelight, Arlo, Ring, ecobee, Lutron Caseta, Netatmo, iHome.
- **Lokalne i otwartoźródłowe środowiska Smart Home:** Home Assistant, Domoticz, OpenHAB, Hubitat Elevation, ioBroker, Jeedom, Zigbee2MQTT, Homey, ZHA (Zigbee Home Automation), Node-RED, HomeBridge.
- **Systemy profesjonalne i przemysłowe:** KNX, Control4, Crestron, Loxone, Fibaro.

Niestety, takie testy są wysoce czasochłonne, a w wielu przypadkach dostęp do tych środowisk uzyskać można jedynie kupując huby producentów, które potrafią być kosztowne.

4.3.2. Test szyfrowania

Bezpieczeństwo jest priorytetem dla dużej grupy konsumentów. Z tego względu należy się upewnić, iż komunikacja sterownika jest szyfrowana, oraz sprawdzić istnienie luk

w zabezpieczeniach (punkt 3.2.2).

Aby przeprowadzić takie badania potrzebny jest sprzęt umożliwiający nasłuchiwanie sygnałów w testowanym protokole, którego obsługa może nie być prosta. Należy też mieć odpowiednie umiejętności i wiedzę czego szukać, by móc potwierdzić bezpieczeństwo z dużą dozą pewności.

4.3.3. Test zabezpieczeń Frame + Nonce

Mimo szyfrowania wiadomości, sterownik nadal może być wrażliwy na ataki Replay (punkt 3.2.2). Symulowanie próby sforsowania zabezpieczeń poprzez wykonywanie ataków samodzielnie wykaże, czy protokoły je zwalczające są poprawnie zaimplementowane.

4.3.4. Test implementacji

Z założenia sterownik uniwersalny powinien być prosty w obsłudze. Dotyczy się to każdego etapu, w tym podpinaniu własnych modułów, programowaniu układu czy nawiązywaniu połączenia z jednostką główną. Wykonanie badania, w którym dano by potencjalnym klientom gotowy moduł do przetestowania, oraz spisano ich reakcje i opinie, dałoby duży wgląd w to, jak końcowy konsument obchodzi się z modułem, i czy ma problemy z jego użytkowaniem.

4.4. Porównanie osiągnięć

Ta sekcja poświęcona jest spisaniu wszystkich parametrów oraz charakterystyk wykonanego prototypu. Została również dokonana analiza porównawcza względem innych urządzeń dostępnych na rynku.

Czyste parametry sterownika (bez podpiętych dodatkowych modułów):

- Wymiary: 60mm / 25mm / 5mm
- Zużycie mocy: 0.3 - 0.5 W
- Możliwość zasilania bateriami: Tak
- Cena: 30 PLN
- Zasięg: 10 - 70 m

- Komunikacja: Zigbee
- W pełni lokalny: Tak
- Kompatybilność: Home Assistant
- Szyfrowane połączenie: Nie
- Pełna dokumentacja: Tak

Choć powyższe dane techniczne są pomocne, pełnię możliwości modułu lepiej ukazuje opis konfiguracji testowej przedstawionej w punkcie 4.2.9. Ceny modułów wzięto z serwisu Allegro.

- Wymiary: 55mm / 80mm / 70mm
- Zużycie mocy: szacowane 0.6 - 0.7 W
- Możliwość zasilania bateriami: Tak
- Cena: 53 PLN
- Pomiar: Temperatura, wilgotność, ciśnienie, naświetlenie
- Wyświetlacz: Tak

Szukając inteligentnych termometrów z wyświetlaczem znaleziono szereg ofert w cenach 30 - 60 PLN, rozmiarowo podobne. Jednak w tej cenie nie zauważono produktu mierzącego również ciśnienie bądź naświetlenie.

Aby trochę lepiej zobrazować parametry sterownika w odniesieniu do innych dostępnych rozwiązań, stworzono tabelę 4.4. **Legenda tabeli 4.4:**

- **Zb** - Zigbee, **Wf** - Wi-Fi, **Th** - Thread, **Bl** - Bluetooth LE.
- **Lock-in** – Poziom vendor-lock-in (Low/Med/High).
- **Sec** - Ocena bezpieczeństwa (Low/Med/High), bazowana na protokole i zgłoszeniach użytkowników.
- **Prc** - Cena detaliczna w Euro (przybliżona wg Allegro / Amazon, czerwiec 2025).
- **Pwr** - Typowe zużycie energii w trybie aktywnym (mW).
- **Size** - XS: <3cm, S: 3-5cm, M: 5-8cm, L: >8cm.

Urządzenie	Zb	Wf	Th	Bl	Lock-in	Sec	Prc	Pwr	Size
ESP32-C6 Custom Module	X				Low	Low	6	120	S
Aqara Motion Sensor	X				High	Med	25	25	XS
Philips Hue White Bulb	X				Med	Med	45	1500	S
Nanoleaf Essentials Bulb	X		X		Med	Med	50	500	S
Tuya Wi-Fi Temp Sensor		X			High	Low	15	800	S
SmartThings Button	X				High	Low	30	150	XS
Xiaomi Mi BLE Thermometer				X	Med	Low	10	100	XS
IKEA TRÅDFRI Remote Switch	X				High	Med	10	60	S
Eve Door/Window Sensor			X		Med	High	40	70	XS
Shelly H&T Wi-Fi Temp+Hum		X			High	Low	22	500	S
Sonoff SNZB-02 Temp Sensor	X				Med	Med	9	60	XS
Innr Zigbee Plug (Outlet)	X				Med	Med	20	800	M
Aqara Wireless Mini Switch	X				High	Med	17	30	XS
Lidl Silvercrest Motion Sensor	X				High	Med	12	40	XS

Tabela 4.4. Porównanie typowych urządzeń końcowych Smart Home względem modułu ESP32-C6

Żadne porównania nie oddadzą jednak głównej zalety wykonanego sterownika uniwersalnego: możliwości rozbudowy i dostosowywania przez użytkownika. Nie ma na rynku innych urządzeń oferujących podobne możliwości. Zachowanie przejrzystości i możliwości zaglądania w kod daje pewność co do bezpieczeństwa.

Ze względu na opisane w następnym punkcie niedogodności, nie udało się wprowadzić jakichkolwiek zabezpieczeń, takich jak szyfrowanie połączenia czy protokoły Frame + Nonce.

Przedstawione wyniki dotyczą wersji prototypowej przygotowanej na potrzeby pracy, jednak nie stanowią ograniczenia końcowych możliwości modułu. Przy zastosowaniu dedykowanego zasilania, optymalizacji firmware'u oraz miniaturyzacji komponentów możliwa jest dalsza poprawa parametrów urządzenia.

4.5. Napotkane problemy

W niniejszym podrozdziale opisano problemy, które wystąpiły na kolejnych etapach realizacji projektu; od konfiguracji środowiska programistycznego, przez integrację protokołu Zigbee, aż po testy i proces debugowania. Sekcja ta przedstawia rzeczywiste trudności napotkane podczas pracy, a tym samym wykazuje istotne problemy, jakie występują w praktyce przy korzystaniu z wymienionych narzędzi.

4.5.1. Początkowe przygotowania

Zgodnie z analizą przedstawioną w punkcie 3.3, wybór mikrokontrolera wspierającego Zigbee był znacząco ograniczony. W praktyce, jedynym dostępnym rozwiązaniem o zadowalających parametrach i cenie było ESP32-C6.

Pierwszym testowanym modulem był ESP32-C6 Super Mini. Ze względu na niewielkie rozmiary wydawał się on idealnym kandydatem do zastosowania w uniwersalnym sterowniku. Niestety, już na etapie uruchomienia pojawiły się poważne problemy. Moduł nie współpracował z żadnym ze środowisk programistycznych, niezależnie od użytego narzędzia (Arduino IDE, ESP-IDF, Visual Studio Code z rozszerzeniami Espressif IDF i PlatformIO). Pomimo licznych prób i konfiguracji, płytką albo nie była rozpoznawana, albo całkowicie odmawiała komunikacji. Jednorazowo udało się wgrać prosty przykład w Arduino IDE, jednak po ponownej próbie (bez żadnych zmian w kodzie) urządzenie przestało działać całkowicie. W związku z tym konieczny był zakup innej płytki. Tym razem postawiono na FireBeetle 2 ESP32-C6, która finalnie posłużyła jako podstawa projektu.

4.5.2. Konfiguracja środowiska

Instalacja środowiska ESP-IDF z oficjalnych źródeł nie była wystarczająca do poprawnej kompilacji projektu. Jednym z problemów był fakt, iż kompilator *ninja*, niezbędny do działania środowiska, nie był domyślnie instalowany ani wskazany jako wymagany. Brak jasnych komunikatów oraz nieaktualna dokumentacja znacząco utrudniały rozpoczęcie pracy. Informacja o konieczności posiadania *ninja* pojawia się jedynie w tabeli zależności technicznych, bez wyróżnienia i bez konkretnego opisu jego roli, co powoduje, że łatwo ją przeoczyć, zwłaszcza przy ręcznej instalacji.

Zintegrowane środowisko Espressif IDE, bazujące na Eclipse, okazało się przestarzałe i mało funkcjonalne. Występowały w nim błędy oznaczeń kodu (false-positive), niezgodne

ze stanem faktycznym kompilatora, a wiele funkcji, takich jak uruchomienie terminala IDF, nie działało mimo stosowania się do oficjalnych poradników. Warto przypomnieć, iż jest to oficjalne IDE, instalowane wraz z całym środowiskiem IDF. Z tego względu edytowanie kodu przeniesiono do Visual Studio Code, a proces kompilacji i wgrywania odbywał się ręcznie przez terminal, co i tak było wymagane w Espressif IDE.

System budowania oparty na *CMake*, już sam w sobie złożony, został dodatkowo utrudniony przez własne rozszerzenia Espressifa. Uniemożliwia to intuicyjne zarządzanie komponentami oraz dodawanie zewnętrznych bibliotek, nawet w przypadku bibliotek oficjalnie udostępnionych w SDK Zigbee. W efekcie, każda zmiana struktury projektu (np. próba jego uproszczenia lub rozdzielenia funkcjonalności) kończyła się błędami.

4.5.3. Integracja Zigbee SDK

Obsługa Zigbee nie jest częścią bazowego ESP-IDF. Konieczne jest ręczne pobranie i dołączenie dodatkowego SDK. Niestety, jego struktura i sposób działania są nieintuicyjne oraz słabo udokumentowane.

Największym problemem była niemożność stworzenia nowego, działającego projektu od podstaw. Mimo wielu prób, nie udało się przygotować czystego projektu, który umożliwiałby kompilację kodu z wykorzystaniem Zigbee SDK. System opiera się silnie na lokalizacji plików, a każda zmiana katalogów powodowała błędy przy załączaniu bibliotek. Nawet oficjalne polecenie `idf.py create-project-from-example`, zaprojektowane do tworzenia nowych projektów na podstawie przykładów, nie działało poprawnie. Przeniesienie przykładu do innego folderu bądź zmiana jego nazwy skutkowały błędami podczas kompilacji. Ostatecznie, kod źródłowy projektu był tworzony bezpośrednio w folderach przykładów, poprzez nadpisywanie istniejących plików. Warto również zaznaczyć, iż próba rozszerzenia przykładu o inną bibliotekę, nawet znajdującą się w tym samym folderze co już dołączona, często kończyło się kolejnymi błędami.

Nawet mając działający projekt, próba rozszerzenia jego funkcjonalności często kończyła się niepowodzeniem. Zigbee SDK jest wrażliwy na każdą zmianę w strukturze programu. Dodanie drugiego endpointu czy własnego klastra powodowało awarie stosu Zigbee (`zb_assert()`). Dokumentacja SDK była niepełna i przestarzała - brakowało przykładów definiowania wielu endpointów, tworzenia własnego klastra oraz listy dostępnych konfiguracji czy typów urządzeń. Jedyna poprawnie działająca metoda zdefiniowania wielu endpointów została znaleziona nie w dokumentacji, a w wątku na GitHubie [60]. Do pisania kodu

i debugowania niezbędnym okazało się narzędzie AI ChatGPT, które znacznie ułatwiło poszukiwanie działających rozwiązań i wyjaśnień enigmatycznych oznaczeń błędów.

4.5.4. Testowanie i debugowanie

Zigbee nie jest protokołem obsługiwany natywnie przez większość komputerów czy telefonów, dlatego konieczne było zastosowanie zewnętrznego koordynatora. Zakupiono dongiel Zigbee (Sonoff ZB Dongle-E) i podjęto próbę konfiguracji Home Assistant na komputerze PC jako środowiska testowego. Niestety, Home Assistant nie działa natywnie w systemie Windows, a konfiguracja z wirtualną maszyną była zawodna - dongiel nie był poprawnie wykrywany.

Ostatecznie, środowisko testowe zostało przeniesione na Raspberry Pi, gdzie po wielu nieudanych próbach udało się skonfigurować dongiel z Home Assistant przy pomocy dodatków Zigbee2MQTT i Mosquitto broker. Mimo to, integracja ESP z Home Assistantem była niestabilna: zmiana konfiguracji endpointów lub klastrów w kodzie często nie była rozpoznawana przez system bez całkowitego usunięcia urządzenia i ponownego procesu parowania. Dodatkowo, testowane urządzenia Zigbee (np. czujnik temperatury Emos Go-Smart) okazały się być zablokowane przez producenta i nie komunikowały się z systemem mimo poprawnego parowania.

Rozdział 5

Analiza otrzymanych wyników w kontekście badań

Ten rozdział skupia się na poszczególnych aspektach wykonanych w ramach pracy. Stanowi spis własnego wkładu w rozwój tematu, znacząco ułatwiający zrozumienie go i dający podstawy do dalszych badań.

5.1. Podsumowanie wyników

Analiza rynku: Aby uzyskać obraz obecnej sytuacji na rynku inteligentnych domów, przeprowadzono szczegółową analizę dostępnych rozwiązań oraz zestawiono je z opiniami konsumentów. Stworzono wykaz najistotniejszych aspektów do implementacji. Na ich podstawie zestawiono przykładowe rozwiązania i porównano ich parametry. Przeanalizowano zagrożenia wynikające z urządzeń podłączonych do internetu oraz systemów chmurowych.

Projekt: Na podstawie wcześniejszych analiz opracowano definicję sterownika uniwersalnego, mającego na celu realizację kluczowych funkcjonalności zgodnych z najistotniejszymi oczekiwaniami konsumentów. Dokonano głębokiej analizy sześciu protokołów komunikacyjnych, wykazując ich wady oraz zalety, a następnie zestawiono je i porównano bezpośrednio. Przedstawiono różne podejścia możliwe do realizacji projektu, zestawiając ich charakterystyki oraz możliwe zastosowania. Na podstawie zebranych informacji podjęto i uzasadniono decyzję co do wykorzystanego rozwiązania dla prototypu. Omówiono możliwe środowiska programistyczne, wymieniając ich wady oraz zalety.

Implementacja i testy: Do przedstawienia faktycznych zastosowań sterownika uniwersalnego stworzono szereg układów demonstracyjnych, które pozwalają na osiągnięcie parametrów porównywalnych z produktami konsumenckimi. Stworzono bibliotekę kodów źródłowych gotowych do użytku, wraz z opisem pozwalającym na ich modyfikację i rozszerzenie. Przeprowadzono dużą liczbę testów, udowadniających działanie prototypu oraz zawierających istotne obserwacje. Na ich podstawie przygotowano porównanie osiągnięć wykonanego rozwiązania na tle innych urządzeń końcowych Smart Home. Opisano problemy, jakie wyniknęły z praktycznego zastosowania wybranego modułu sprzętowego oraz środowiska programistycznego.

5.2. Ograniczenia proponowanego rozwiązania

Pomimo udanej implementacji i szerokiego zakresu funkcjonalności, zaprezentowane rozwiązanie nie jest wolne od ograniczeń. Wynikają one zarówno z przyjętych założeń projektowych, jak i z aspektów technicznych zastosowanych narzędzi oraz środowiska, które zostały dokładnie omówione w punkcie 4.5.

- **Brak implementacji zabezpieczeń:** W obecnej wersji nie zaimplementowano szyfrowania transmisji Zigbee, licznika ramek ani żadnych zabezpieczeń podczas procesu parowania. Wynika to z ograniczeń biblioteki Zigbee SDK dostarczanej przez Espressif, niedostatecznej dokumentacji oraz wysokiego progu wejścia w implementację niskopoziomową. W pełnej wersji produkcyjnej takie mechanizmy powinny zostać uwzględnione, gdyż są kluczowe do utrzymania bezpieczeństwa.
- **Brak wsparcia dla pełnego zestawu klastrów:** Obecny prototyp wykorzystuje ograniczoną liczbę klastrów ZCL (np. On/Off, Temperature Measurement). Choć przesył danych jest technicznie możliwy, jego realizacja jest nieporęczna i nieczytelna. Rozszerzenie urządzenia o dodatkowe klastry, takie jak sterowanie jasnością, energią czy powiadomienia IAS, wymaga ręcznej konfiguracji i głębszej znajomości specyfikacji Zigbee. Końcowa wersja urządzenia powinna móc przysyłać dane w przejrzystej formie, z ich prawidłowym opisem.
- **Zależność od środowiska Home Assistant:** Chociaż urządzenie może teoretycznie współpracować z każdym koordynatorem Zigbee, testy przeprowadzono jedynie w środowisku Home Assistant i Zigbee2MQTT. Integracja z zamkniętymi ekosystemami (np. Tuya, SmartThings) nie została zweryfikowana i może wymagać dodat-

kowych adaptacji.

- **Złożoność procesu konfiguracji:** Użytkownik końcowy musi samodzielnie skompilować i wgrać oprogramowanie oraz podłączyć wybrane komponenty peryferyjne. Brak interfejsu graficznego lub systemu automatycznej konfiguracji sprawia, że prototyp kierowany jest głównie do użytkowników zaawansowanych technicznie. Wymagane środowisko Espressif IDF jest trudne do konfiguracji i użytkowania, co znacznie ogranicza możliwość rozszerzenia funkcjonalności.
- **Rozmiar fizyczny oraz zużycie energii:** Chociaż prototyp wykazuje dobrą efektywność energetyczną w porównaniu do innych rozwiązań, jego zużycie mocy może być ograniczeniem w przypadku długoterminowego zasilania bateryjnego. Rozmiar bazowej wersji sterownika jest stosunkowo duży, co ogranicza możliwości jego zastosowania, zwłaszcza w kontekście konieczności dołączania dodatkowych modułów, które jeszcze bardziej zwiększają wymiary.

Opisane ograniczenia nie unieważniają funkcjonalności zaprezentowanego sterownika, ale wskazują obszary, które powinny zostać rozwinięte w przyszłych iteracjach projektu.

5.3. Możliwości dalszego rozwoju

Opracowany prototyp stanowi solidną podstawę, jednak istnieje szereg kierunków, w których projekt może być znacząco rozwinięty:

- **Własna płytką PCB:** Stworzenie dedykowanego projektu płytki drukowanej pozwoliłoby znacząco zmniejszyć rozmiar urządzenia, zoptymalizować układ peryferiów oraz obniżyć zużycie energii. Umożliwiłoby to również lepszą kontrolę nad zasilaniem i stabilnością pracy układu.
- **Nowe środowisko programistyczne:** Aktualny system oparty o Espressif IDF cechuje się wysokim progiem wejścia oraz ograniczoną ergonomią. Alternatywnym podejściem mogłoby być stworzenie własnego środowiska lub biblioteki, bazującej np. na języku C++ z podejściem obiektowym, co zwiększyłoby czytelność kodu oraz ułatwiło jego rozwój. Dobrym punktem odniesienia jest tutaj platforma Arduino.
- **Wsparcie dla programowania graficznego:** Wdrożenie warstwy programistycznej w formie edytora blokowego (np. Node-RED lub Blockly) umożliwiłoby tworzenie

logiki działania urządzenia w sposób bardziej przystępny dla użytkowników nieposiadających doświadczenia z programowaniem. Rozwiązanie to znacząco zwiększyłoby dostępność projektu.

- **Implementacja pełnych zabezpieczeń:** Obecna wersja prototypu nie implementuje mechanizmów bezpieczeństwa takich jak szyfrowanie transmisji, uwierzytelnianie czy kontrola ramek. Ich wdrożenie wymaga głębokiej znajomości SDK Zigbee oraz specyfikacji ZCL, co powinno zostać uwzględnione w przyszłych wersjach. Zaimplementowanie pełnego stosu zabezpieczeń jest kluczowe dla zapewnienia ochrony danych w środowisku Smart Home.
- **Rozszerzenie obsługi klastrów ZCL:** Aktualna obsługa ogranicza się do podstawowych klastrów. Pełna implementacja dodatkowych funkcji takich jak np. raportowania energii, scen czy alarmów IAS, wymagałaby rozbudowy warstwy aplikacyjnej i ręcznej rejestracji atrybutów.
- **Zmiana mikrokontrolera lub protokołu:** W przypadku trudności z dalszym rozwijaniem rozwiązania na bazie ESP32-C6, warto rozważyć użycie innego mikrokontrolera bądź systemu SoC, oferującego lepszą dokumentację lub wsparcie. W razie potrzeby możliwa jest również zmiana protokołu, np. na Thread lub Bluetooth. Komunikacja Wi-Fi, z uwagi na wysoki pobór energii, brak natywnej struktury sieciowej, oraz potencjalne zagrożenia płynące z podłączenia do internetu jest najmniej korzystnym wyborem dla urządzeń końcowych.

Kierunkiem rozwoju zapewniającym najlepsze perspektywy jest stworzenie własnego PCB od podstaw, wraz z odpowiednio zaprojektowanym środowiskiem programistycznym. Alternatywnym, prostszym podejściem może być zmiana mikrokontrolera na taki, który oferuje bardziej przyjazne narzędzia deweloperskie. W przypadku konieczności pozostania przy Espressif IDF, niezbędne będzie poświęcenie czasu na głęboką analizę oraz opracowanie własnej dokumentacji ułatwiającej dalsze prace.

5.4. Wnioski końcowe

Na podstawie przeprowadzonych badań, testów oraz analizy wyników można sformułować następujące wnioski:

Zarówno obecni, jak i potencjalni użytkownicy systemów Smart Home wyrażają wyraźne obawy związane z prywatnością, bezpieczeństwem oraz brakiem interoperacyjności pomiędzy urządzeniami różnych producentów. Komfort użytkowania i niezawodność integracji plasują się wśród najważniejszych oczekiwań konsumentów, a ich spełnienie nadal stanowi istotne wyzwanie na rynku.

Gotowe rozwiązania komercyjne, mimo atrakcyjnej estetyki i popularności, często wiążą się z wysoką ceną, niskimi możliwościami rozbudowy oraz zagrożeniami bezpieczeństwa wynikającymi z połączenia z internetem. Ich ograniczona kompatybilność oraz zamknięte środowiska stanowią przeszkodę dla świadomych użytkowników.

Protokół Zigbee, dzięki niskiemu zużyciu energii, obsłudze komunikacji mesh oraz wbudowanym mechanizmom zabezpieczeń, stanowi solidną podstawę do realizacji rozwiązań lokalnych, zwłaszcza przy założeniu pełnej kontroli nad infrastrukturą.

Wykorzystanie gotowych płytek deweloperskich, takich jak ESP32-C6, okazało się wystarczające na etapie prototypowania, lecz w perspektywie rozwoju projektu do etapu produkcji masowej, bądź do osiągnięcia optymalnych parametrów, takich jak rozmiar, zużycie energii oraz wygoda użytkowania, sugerowane jest rozważenie konstrukcji własnej płytki PCB oraz opracowanie przyjaznego środowiska programistycznego.

Udowodniono, że możliwe jest stworzenie sterownika uniwersalnego, który zapewnia funkcjonalność porównywalną z popularnymi urządzeniami końcowymi dostępnymi na rynku. Jednocześnie oferuje większą przejrzystość, niższy koszt produkcji oraz potencjalnie wyższy poziom prywatności i kontroli, z możliwością rozbudowy oraz dużo lepszego dopasowania funkcjonalności do potrzeb indywidualnego użytkownika.

Projekt może również pełnić funkcję edukacyjną, stanowiąc bazę do nauki programowania systemów wbudowanych, analizy protokołów komunikacyjnych oraz projektowania własnych rozwiązań Smart Home, wspierając tym samym rozwój otwartej społeczności technologicznej i IoT (Internet of Things).

Opracowane rozwiązanie pokazuje, że lokalna, otwarta infrastruktura Smart Home może stanowić realną alternatywę dla popularnych systemów opartych o chmurę, eliminując wiele problemów związanych z prywatnością, bezpieczeństwem oraz brakiem elastyczności.

Doświadczenia zdobyte podczas implementacji zwracają uwagę na ograniczenia dostępnych narzędzi deweloperskich, szczególnie w kontekście dokumentacji i wygody użytkowania. Sugeruje to potrzebę opracowania bardziej przyjaznego środowiska lub wyboru alternatywnych platform dla przyszłych iteracji projektu.

Pomimo udanego wdrożenia, projekt ujawnia szereg obszarów wymagających dalszego rozwoju, szczególnie w zakresie bezpieczeństwa, kompatybilności z dodatkowymi klastrami Zigbee oraz uproszczenia procesu konfiguracji. Tym samym zaprezentowane rozwiązanie stanowi wartościowy fundament dla dalszych prac badawczo-rozwojowych w obszarze otwartych systemów Smart Home.

Podsumowanie: Zaprezentowane rozwiązanie stanowi nie tylko przykład działającego sterownika, ale także propozycję podejścia do projektowania otwartych, bezpiecznych i rozszerzalnych urządzeń końcowych w standardzie Zigbee, czego nie oferują rozwiązania komercyjne. Tym samym praca wnosi istotny wkład w rozwój niezależnych i otwartych rozwiązań Smart Home.

Bibliografia

- [1] IoT Insider. Smart home devices are an easy backdoor for cyber attackers.
<https://www.iotinsider.com/iot-insights/technical-insights/smart-home-devices-are-an-easy-backdoor-for-cyber-attackers>, 2024.
- [2] B. A. Edem et al. Cybersecurity threats and challenges of smart home systems.
<https://www.scribd.com/document/864909466/1-s2-0-S2949715923000793>, 2023. Originally published in: Smart Systems and Sustainability, Elsevier.
- [3] E-SPIN Corporation. Exploring zigbee network topology: Unveiling the three types for robust communication.
<https://www.e-spincorp.com/zigbee-network-topology/>, 2023.
- [4] Connectivity Standards Alliance. Zigbee frequently asked questions.
<https://csa-iot.org/all-solutions/zigbee/>, 2024.
- [5] Wi-Fi Alliance. Discover wi-fi. <https://www.wi-fi.org/discover-wi-fi>, 2024.
- [6] Bluetooth SIG. Bluetooth core specification 5.4.
<https://www.bluetooth.com/specifications/specs/core-specification-5-4/>, 2023.
- [7] Connectivity Standards Alliance. Zigbee frequently asked questions.
<https://csa-iot.org/all-solutions/zigbee/>, 2024.
- [8] Espressif Systems. Esp zigbee mesh examples and capabilities.
<https://github.com/espressif/esp-zigbee-sdk>, 2024.
- [9] DFRobot. Wi-fi vs bluetooth vs zigbee: A comparison guide.
<https://www.dfrobot.com/blog-1693.html>, 2023.
- [10] Silicon Labs. Mesh networking technologies.
<https://www.silabs.com/wireless/mesh-networking>, 2023.

- [11] Z-Wave Alliance. Z-wave specification and benefits. <https://z-wavealliance.org>, 2023.
- [12] Z-Wave Alliance. Z-wave specification and benefits. <https://z-wavealliance.org>, 2023.
- [13] Thread Group. Thread technology overview. <https://www.threadgroup.org/What-is-Thread>, 2024.
- [14] Thread Group. What is thread? <https://www.threadgroup.org/What-is-Thread>, 2024.
- [15] Wired. Matter smart home standard explained. <https://www.wired.com/story/matter-smart-home-explainer/>, 2023.
- [16] Harold Ablowitz and Chris Hunt. *Smart Homes and Their Users*. Springer, 2013.
- [17] Eurostat. Digital economy and society statistics - households and individuals. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Digital_economy_and_society_statistics_-_households_and_individuals, 2024.
- [18] Exploding Topics. Smart home stats: 2024 industry trends. <https://explodingtopics.com/blog/smart-home-stats>, 2024.
- [19] GreenMatch UK. Smart home statistics in 2024. <https://www.greenmatch.co.uk/blog/smart-home-statistics>, 2024.
- [20] Strategic Market Research. Smart home statistics and trends 2024. <https://www.strategicmarketresearch.com/blogs/smart-home-statistics>, 2024.
- [21] YouGov. Us smart home report. <https://commercial.yougov.com/rs/464-VHH-988/images/WP-2024-01-YouGov-US-Smart-Home-Report.pdf>, 2023.
- [22] Morning Consult. What consumers want (and don't want) in smart-home tech. <https://pro.morningconsult.com/analysis/smart-home-tech-consumer-preferences>, 2024.
- [23] Coolest Gadgets / Statista. Global smart speaker market report 2024. <https://www.globenewswire.com/news-release/2023/08/14/...>, 2024.

- [24] Eurostat. Digital economy and society statistics – households and individuals. https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Digital_economy_and_society_statistics_-_households_and_individuals, 2023.
- [25] GlobeNewswire. Global cleaning robot market report 2023. <https://www.globenewswire.com/news-release/2023/03/08/...>, 2023.
- [26] Statista. Smart home statistics and trends. <https://www.statista.com/...>, 2023.
- [27] Deloitte. Connected consumer survey 2023. <https://www2.deloitte.com/...>, 2023.
- [28] HealthPopuli. Consumers continue to spend on technology, seeking “a happy, healthy connected life”. <https://www.healthpopuli.com/2023/09/11/consumers-continue-to-spend-on-technology-seeking-a-happy-healthy-connected-life>, 2023.
- [29] Sonoff. Sterownik sonoff zbmini zigbee. <https://allegro.pl/oferta/sterownik-sonoff-zbmini-zigbee-son-17372260223>, 2025.
- [30] Shelly. Shelly 1 gen3 - przekaźnik wifi/bluetooth. <https://botland.com.pl/shelly-automatyka-domowa/25604-shelly-1-gen3-przekaznik-12v24v-48vdc240vac-wifibluetooth-16a-aplikacja-androidios-3800235261767.html>, 2025.
- [31] Fibaro. Fibaro fgs-213 z-wave+ switch 2-kanalowy. <https://allegro.pl/oferta/fibaro-fgs-213-zw5-sterownik-z-wave-2-kanalowy-16588995544>, 2025.
- [32] Aqara. Czujnik temperatury i wilgotności aqara. <https://allegro.pl/oferta/czujnik-temperatury-i-wilgotnosci-aqara-3-6x3-6x0-9-cm-bialy-17610300071>, 2025.
- [33] Shelly. Shelly h&t - czujnik temperatury i wilgotności. <https://www.komputronik.pl/product/962156/shelly-h-t-bialy-.html>, 2025.
- [34] Fibaro. Czujnik ruchu pir z-wave. <https://allegro.pl/oferta/czujnik-ruchu-pir-z-wave-fibaro-smarthings-17279898222>, 2025.
- [35] Philips Hue. Żarówka philips hue 9w a60 e27 (2-pack). <https://allegro.pl/oferta/philips-hue-zarowka-9w-a60-e27-2-pack-14476736913>, 2025.

- [36] Google Nest. Nest protect - smart smoke detector (wired).
<https://www.coolshop.pl/produkt/google-nest-protect-smart-smoke-detector-wired-powersource-dk-no/234PF6/>, 2025.
- [37] Gosund. Gniazdko wifi gosund sp111 z watomierzem.
<https://allegro.pl/oferta/gniazdko-wifi-gosund-3680w-16a-tuya-sp111-timer-programator-watomierz-15458510543>, 2025.
- [38] The Guardian. Amazon's ring cameras hacked to threaten users, lawsuit claims.
<https://www.theguardian.com/technology/2020/dec/23/amazon-ring-camera-hack-lawsuit-threats>, 2020.
- [39] GBHackers. Smart bed security flaw leaks sensitive data and access to home network. <https://gbhackers.com/smart-bed-security-flaw/>, 2024.
- [40] Truffle Security. Removing jeff bezos from my bed.
<https://trufflesecurity.com/blog/removing-jeff-bezos-from-my-bed>, 2024.
- [41] The Guardian. Google home smart speakers record and send audio to google, reports reveal. <https://www.theguardian.com/technology/2019/jul/11/google-home-assistant-listen-recordings-users-privacy>, 2019.
- [42] VOA News. Cybersecurity experts worried about chinese firms' control of smart devices. https://www.voanews.com/a/east-asia-pacific_voa-news-china_cybersecurity-experts-worried-chinese-firms-control-smart-devices/6209815.html, 2021.
- [43] ASSET Research Group. Sweyntooth: Unleashing mayhem over bluetooth low energy. <https://asset-group.github.io/disclosures/sweyntooth/>, 2020.
- [44] KNOB Research Team. Knob attack – key negotiation of bluetooth.
<https://knobattack.com/>, 2019.
- [45] CISA. Sweyntooth vulnerabilities alert. <https://www.cisa.gov/news-events/alerts/2020/03/06/sweyntooth-vulnerabilities>, 2020.
- [46] Bluetooth SIG. Bluetooth le secure connections. <https://www.bluetooth.com/blog/bluetooth-security-le-secure-connections/>, 2016.
- [47] Jun Yun and Stefan Schschneider. Building automation systems: The case of thread.
<https://www.diva-portal.org/smash/get/diva2:1268444/FULLTEXT01.pdf>, 2018.

- [48] IETF. Rfc 7925 – tls/dtls profiles for iot.
<https://datatracker.ietf.org/doc/html/rfc7925>, 2016.
- [49] Connectivity Standards Alliance. Matter security and privacy fundamentals.
<https://csa-iot.org/wp-content/uploads/2023/01/Matter-Security-and-Privacy-Whitepaper.pdf>, 2023.
- [50] Nozomi Networks. Trust matters: Vulnerabilities in matter. <https://www.nozominetworks.com/blog/trust-matters-vulnerabilities-in-matter/>, 2024.
- [51] Project CHIP. Matter reference implementation.
<https://github.com/project-chip/connectedhomeip>, 2024.
- [52] Silicon Labs. Z-wave s2 security. <https://www.silabs.com/documents/public/application-notes/AN1274-Z-Wave-S2-Application-Note.pdf>, 2018.
- [53] Silicon Labs. Introduction to z-wave smartstart.
<https://www.silabs.com/documents/public/application-notes/AN0428.pdf>, 2017.
- [54] Exploitee.rs. Z-wave exploit summary.
https://www.exploitee.rs/index.php/Z-Wave_Exploit, 2020.
- [55] Wenyuan Wang et al. Insecure to the touch: Attacking zigbee 3.0 via touchlink commissioning. <https://www.eurecom.fr/publication/5743>, 2017.
- [56] Tobias Zillner. Zigbee exploited: The good, the bad and the ugly.
<https://media.blackhat.com/us-15/US-15-Zillner-ZigBee-Exploited-WP.pdf>, 2015.
- [57] Silicon Labs. Zigbee security best practices (an1233). <https://www.silabs.com/documents/public/application-notes/an1233-zigbee-security.pdf>, 2020.
- [58] Devu Shila et al. Ghost-in-the-wireless: Energy depletion attack on zigbee.
<https://arxiv.org/abs/1405.0084>, 2014.
- [59] Zigbee2MQTT contributors. Zigbee network.
https://www.zigbee2mqtt.io/advanced/zigbee/01_zigbee_network.html, 2025.

[60] Espressif Systems. Multiple endpoints (tz-1070).

<https://github.com/espressif/esp-zigbee-sdk/issues/407>, March 2022.

GitHub Issue #407.

Źródła zostały sprawdzone w czerwcu 2025. Do zbiorów danych i analizy rynku częściowo wykorzystane zostało narzędzie AI ChatGPT.